Academic Section & Admin Management System

User Login & Authentication Service

Service Documentation — System Design

1 Service Overview

The User Login & Authentication Service is responsible for verifying the identity of Students, Faculty, and Administrators using their Institute IDs. It acts as the primary gatekeeper of the system, handling LDAP integration for credential verification, determining Role-Based Access Control (RBAC) levels from the database, and issuing secure JSON Web Tokens (JWT) to maintain stateless, secure sessions across all system microservices.

2 Core Responsibilities

- **Identity Verification:** Authenticate users via the institutional LDAP server.
- **Role Assignment:** Fetch and assign precise RBAC roles (Student, Faculty, Admin, Staff).
- **Token Generation:** Create secure JWTs with signed payload data for downstream authorization.
- **Session State Management:** Facilitate secure cookie generation for persistence.
- **Audit Logging:** Log all successful and failed login attempts securely.

3 Architecture & Components

3.1 Internal Modules

- **AuthController:** Parses login requests and coordinates the authentication flow.
- **AuthService:** Core orchestrator handling the business logic of logging in.
- **LDAPConnector:** Interfaces with the external directory for password verification.
- **JWTGenerator:** Generates, signs, and decodes JWTs using private keys.
- **UserRepository:** Handles database queries to fetch user roles and profiles.
- **AuditLogger:** Encrypts and stores access events in the activity logs database.

4 Authentication Lifecycle

State	Trigger	Description
Requested	Login attempt	User submits Institute ID and Password.
LDAP-Verified	Password match	Credentials validated against institution records.
Token-Issued	RBAC fetched	DB returns user role; JWT is generated and signed.
Active	Valid JWT	User accesses protected routes using the token.
Revoked	Logout / Timeout	Token is cleared from client; session ends.

5 API Endpoints

Method	Endpoint	Description
POST	/api/auth/login	Authenticate credentials and return JWT.
GET	/api/auth/verify	Validate JWT validity for frontend routing.
POST	/api/auth/logout	Clear authentication cookies/tokens.

6 Data Model

6.1 User Database Entry (MySQL)

Field	Type	Description
id	UUID	Internal unique identifier.
institute_id	VARCHAR	Public institutional ID (e.g., Roll number / Employee ID).
role	ENUM	System role (Admin, Faculty, Student, Staff).
is_active	BOOLEAN	Flag determining if the account is currently enabled.
last_login	TIMESTAMP	Record of the most recent successful authentication.

7 Security Considerations

- **JWT Security:** Tokens signed with HS256/RS256 algorithms. No sensitive data in payload.
- **Transmission:** Strict HTTPS enforcement to prevent packet sniffing.
- **Storage:** JWTs stored in HTTP-Only, Secure cookies to mitigate XSS attacks.
- **Password Safety:** Passwords are never stored locally; passed securely to LDAP.
- **Auditing:** All system access modifications are tracked via encrypted Activity Logs.

8 Inter-Service Integration

1. Client submits login form to API Gateway.
2. Gateway routes request to Auth Service.
3. Auth Service queries LDAPConnector for credential validity.
4. Auth Service queries UserRepository for RBAC details.
5. Auth Service returns Signed JWT via HTTP-only cookie.

9 Error Handling

Error Code	HTTP	Description
INVALID_CREDENTIALS	401	Incorrect Institute ID or password.
LDAP_UNAVAILABLE	503	Cannot reach institutional directory service.
ACCOUNT_DISABLED	403	User role is deactivated in the system.
MALFORMED_REQUEST	400	Missing login payload fields.

10 Configuration Parameters

Parameter	Default	Description
JWT_SECRET_KEY	-	Cryptographic key for signing tokens.
JWT_EXPIRY_HOURS	12	Max lifespan of a generated access token.
LDAP_SERVER_URL	ldap://local	Endpoint for the institutional directory.
MAX_FAILED_ATTEMPTS	5	Brute-force protection threshold.

11 Sequence Diagram Observations

(Diagrams maintained externally)

- **Login Flow:** The client sends credentials to the AuthController. It delegates LDAP validation. Upon success, the database is queried for the user's role. A JWT is minted and returned to the client as an HttpOnly cookie, while the AuditLogger records a `LOGIN_SUCCESS` event asynchronously.
- **Failed Attempt Flow:** If LDAP rejects the credentials, the AuthService immediately halts the flow, returning a 401 Unauthorized status and firing a `LOGIN_FAILED` audit event to monitor for brute-force attacks.

12 Class Diagram Responsibilities

(Diagrams maintained externally)

- **AuthController:** Thin entry point. Parses login requests (Institute ID, password) and delegates strictly to AuthService.
- **AuthService:** Core orchestrator. Calls LDAPConnector for verification, UserRepository for roles, and JWTGenerator for the token.
- **User (Entity):** Pure data model representing the database schema (id, institute_id, role).
- **LDAPConnector:** Abstraction for directory binding. Encapsulates network logic and credential transmission.
- **JWTGenerator:** Handles standard cryptographic token generation.
- **AuditLogger:** Fire-and-forget logger handling encrypted tracking of login metrics.

Academic Section & Admin Management System

Password Management & Credential Recovery Service

Service Documentation — System Design

1 Service Overview

The Password Management Service provides a secure, self-service mechanism for users (Students, Faculty, Admins) to recover their accounts and reset their passwords. It manages the generation of time-limited cryptographic reset tokens, coordinates with the Email Communication Service to deliver recovery links, and ensures that new passwords are encrypted using strong hashing algorithms (Bcrypt) before updating the system directory.

2 Core Responsibilities

- **Token Generation:** Create secure, unpredictable, and time-bound password reset tokens.
- **Email Dispatch:** Interface with the email service to deliver recovery links to registered email addresses.
- **Token Validation:** Ensure tokens are valid, unexpired, and have not been previously used.
- **Password Hashing:** Securely hash new passwords using Bcrypt before storage.
- **Credential Synchronization:** Update the central directory (Database/LDAP) with the new credentials.
- **Audit Logging:** Log password reset requests and completions to monitor for suspicious account takeover attempts.

3 Architecture & Components

3.1 Internal Modules

- **PasswordController:** Handles incoming HTTP requests for forgotten and reset passwords.
- **PasswordService:** Orchestrates the logic for verifying users, generating tokens, and updating passwords.
- **TokenRepository:** Manages the persistence and state (used/unused) of reset tokens in the database.
- **EmailService:** Handles SMTP connections to send HTML formatted recovery emails.
- **CryptoManager:** Utility for hashing passwords (Bcrypt) and generating cryptographically secure token strings.
- **UserRepository:** Interfaces with the database to locate user records and apply password updates.

4 Password Reset Lifecycle

State	Trigger	Description
Requested	User submits email	System verifies email exists and generates a reset token.
Sent	Token generated	System emails the token link to the user.
Pending	User clicks link	User is prompted to enter a new password on the frontend.
Consumed	Form submitted	Token is validated, marked as used, and password is updated.
Expired	Time elapsed	If unused within 15 mins, the token becomes permanently invalid.

5 API Endpoints

Method	Endpoint	Description
POST	/api/password/forgot	Accepts email/ID, generates token, triggers email.
GET	/api/password/verify	Validates token status when user opens the email link.
POST	/api/password/reset	Accepts token and new password, updates credentials.

6 Data Model

6.1 Password Reset Token Entry (MySQL)

Field	Type	Description
token_id	UUID	Primary key for the token record.
user_id	UUID	Foreign key linking to the User requesting the reset.
hashed_token	VARCHAR	SHA-256 hash of the token string (for security).
expires_at	TIMESTAMP	Exact time the token becomes invalid.
is_used	BOOLEAN	Flag to ensure single-use (prevents replay attacks).

7 Security Considerations

- **Token Hashing:** The actual token string is sent to the user, but only its hash is stored in the database. If the DB is compromised, tokens cannot be used.
- **Single-Use Enforcement:** Once a password is reset, the ‘is_used‘ flag is flipped, immediately invalidating the token.
- **Rate Limiting:** Strict rate limits (e.g., 3 requests per hour) on the ‘/forgot‘ endpoint to prevent email spam and user enumeration attacks.
- **Password Strength Enforced:** Backend validation ensuring Minimum 8 characters, alphanumeric, and symbols.
- **Timing Attacks:** Uniform response times for the ‘/forgot‘ endpoint regardless of whether the email exists in the system to prevent email enumeration.

8 Inter-Service Integration

1. Client submits "Forgot Password" form.
2. PasswordService queries UserRepository to verify user existence.
3. PasswordService generates token and saves hash via TokenRepository.
4. PasswordService triggers EmailService (SMTP) to send the recovery URL.
5. On reset, PasswordService updates the DB/LDAP via UserRepository and logs the event via AuditLogger.

9 Error Handling

Error Code	HTTP	Description
TOKEN_INVALID	400	Token is malformed, does not exist, or signature fails.
TOKEN_EXPIRED	401	Token has passed its TTL (Time-To-Live).
TOKEN_ALREADY_USED	403	Token has already been consumed for a reset.
WEAK_PASSWORD	422	Provided password does not meet security policies.

10 Configuration Parameters

Parameter	Default	Description
RESET_TOKEN_EXPIRY_MINS	15	Time in minutes before a reset token expires.
BCRYPT_SALT_ROUNDS	12	Work factor for hashing the new password.
SMTP_HOST	smtp.mail	Server address for outbound recovery emails.
RATE_LIMIT_FORGOT_PWD	3	Max forgot password requests per user per hour.

11 Sequence Diagram Observations

(Diagrams maintained externally)

- **Forgot Password Flow:** The system protects against enumeration by always returning a generic "If an account exists, an email was sent" message. Behind the scenes, the token generation and email dispatch are handled asynchronously.
- **Reset Execution Flow:** During the final reset, the service first verifies the token hasn't expired or been used. It then salts and hashes the new password via Bcrypt, updates the database, invalidates the token, and writes to the audit log in a single transactional block to ensure data consistency.

12 Class Diagram Responsibilities

(Diagrams maintained externally)

- **PasswordController:** Parses '/forgot' and '/reset' HTTP requests and handles HTTP responses.

- **PasswordService:** Contains core business logic. Generates tokens, triggers emails, and coordinates the password update process.
- **TokenRepository:** Handles CRUD operations for the `PasswordResetToken` database table.
- **CryptoManager:** Utility class abstracting Bcrypt hashing and random secure string generation.
- **EmailService:** Handles the formatting and transmission of recovery emails to the user.

Academic Section & Admin Management System

Role-Based Access Control (RBAC) Service

Varshith Grandhi
Roll No: 230101041

Service Documentation — System Design

1 Service Overview

The Role-Based Access Control (RBAC) Service acts as the security enforcement layer across the entire application ecosystem. Operating primarily as middleware, it intercepts incoming API requests, extracts the user's identity and assigned role from their JSON Web Token (JWT), and evaluates their permissions against the requested resource. It ensures that Students, Faculty, Wardens, and Admins can only access modules explicitly authorized for their role.

2 Core Responsibilities

- **Request Interception:** Act as a gatekeeper for all protected API routes.
- **Identity Extraction:** Decode and validate the JWT to securely extract the user's role without requiring a database trip on every request.
- **Permission Evaluation:** Match the extracted role against the required permissions for the specific route or resource.
- **Enforcement:** Allow valid requests to proceed (`next()`) or block unauthorized access (`403 Forbidden`).
- **Security Auditing:** Log denied access attempts to monitor for privilege escalation attacks or compromised accounts.

3 Architecture & Components

3.1 Internal Modules

- **RBACMiddleware:** The core interceptor function injected into Express.js/Node.js routes.
- **PermissionService:** Evaluates access rules by mapping roles to specific system resources and actions (Read, Write, Update, Delete).
- **TokenValidator:** Verifies the cryptographic signature of the JWT to ensure the role claim has not been tampered with.
- **RoleRegistry:** An in-memory cache or database table storing the hierarchical mapping of roles and their allowed endpoints.
- **AuditLogger:** Records security events, specifically unauthorized access attempts.

4 Access Evaluation Lifecycle

State	Trigger	Description
Intercepted	API Request	Middleware catches the incoming HTTP request.
Token-Parsed	Auth Header read	The Bearer token is extracted and verified cryptographically.
Evaluated	Route Match	User's role from the token is checked against route requirements.
Authorized	Match Success	Request is passed to the downstream controller.
Denied	Match Failure	Request is terminated; a 403 response is returned to the client.

5 API Endpoints

Method	Endpoint	Description
GET	`/api/rbac/my-permissions`	Returns a list of UI components the current user can view.
GET	`/api/rbac/roles`	(Admin Only) List all system roles and inherited permissions.
PUT	`/api/rbac/roles/update`	(Admin Only) Modify dynamic permissions for a specific role.

6 Data Model

6.1 Role-Permission Mapping (MySQL)

Field	Type	Description
`role_name`	VARCHAR	Identifier for the role (e.g., 'STUDENT', 'FACULTY', 'ADMIN').
`resource`	VARCHAR	The module being accessed (e.g., 'GRADES', 'LEAVE_REQUESTS').
`action`	ENUM	Permitted operation: CREATE, READ, UPDATE, DELETE.
`is_active`	BOOLEAN	Allows temporary disabling of certain permissions.

7 Security Considerations

- **Fail-Secure Default:** If a route does not have an explicit RBAC policy defined, the system defaults to *Deny All*.
- **Tamper-Proof Roles:** Roles are embedded in the JWT payload and signed. If a user alters their token to elevate their role, the signature validation will fail.
- **Hierarchical Permissions:** Admins automatically inherit the read permissions of lower-tier roles to prevent redundant database entries.
- **Privilege Escalation Prevention:** Strict separation between Student logic and Admin logic at the routing level.

8 Inter-Service Integration

1. Client makes an API request to a protected endpoint (e.g., `POST /api/hostel/leave`).
2. The API Gateway routes the request through the `RBACMiddleware`.
3. `RBACMiddleware` calls `TokenValidator` to verify the JWT and extract the role ('STUDENT').
4. `RBACMiddleware` calls `PermissionService` to check if 'STUDENT' can perform 'CREATE' on 'LEAVE_REQUESTS'.
5. Upon success, the request proceeds to the Hostel Management Controller.

9 Error Handling

Error Code	HTTP	Description
`MISSING_TOKEN`	401	No Authorization header was provided in the request.
`INVALID_TOKEN`	401	JWT signature is invalid or token has expired.
`INSUFFICIENT_ROLE`	403	The user's role is valid but lacks permissions for the action.
`RESOURCE_LOCKED`	403	The resource requires SuperAdmin privileges.

10 Configuration Parameters

Parameter	Default	Description
`ENFORCE_STRICT_RBAC`	true	If false, bypasses role checks (for local dev only).
`SUPERADMIN_ROLE`	`SUPER_ADMIN`	Role string that bypasses all permission checks.
`CACHE_PERMISSIONS_TTL`	3600	Seconds to cache role mappings in memory (Redis).

11 Sequence Diagram Observations

- **Successful Authorization:** The RBAC middleware operates entirely in memory (via JWT payload and cached role configurations) to minimize latency. It successfully extracts the role, validates it against the route requirement, and calls `next()` to pass execution to the controller.

- **Unauthorized Access Attempt:** If a 'STUDENT' attempts to hit an 'ADMIN' route (e.g., System Configuration), the `PermissionService` immediately identifies the mismatch. The middleware terminates the HTTP request, sends a 403 Forbidden status, and triggers a critical event to the `AuditLogger`.

12 Class Diagram Responsibilities

- **RBACMiddleware:** The entry point for the authorization layer. Wraps Express routes with `requireRole(role)` or `requirePermission(resource, action)` logic.

- **PermissionService:** Evaluates business logic. Understands the mapping between user roles and system resources.

- **TokenValidator:** Dedicated class for checking JWT integrity, extracting the payload, and handling expiration edge cases.
- **RoleRegistry:** Interface to fetch the active permission matrix. Usually backed by Redis for high performance.
- **AuditLogger:** Records all instances of 403 Forbidden errors to track potential malicious actors.

Academic Section & Admin Management System

User Profile Management Service

Service Documentation — System Design

1 Service Overview

The User Profile Management Service allows users (Students, Faculty, and Staff) to securely view and update their personal information. While core academic or institutional data (e.g., Institute ID, official email, department) is read-only and managed by administrators, this service empowers users to manage mutable data such as contact numbers, residential addresses, and emergency contact details. It ensures that all profile updates are validated, sanitized, and securely logged.

2 Core Responsibilities

- **Data Retrieval:** Fetch and aggregate user profile data from the central database based on the authenticated user's JWT.
- **Data Modification:** Process and apply updates to permissible profile fields (e.g., phone number, local address).
- **Input Sanitization:** Validate user inputs to prevent XSS (Cross-Site Scripting) and SQL injection attacks.
- **Access Control Enforcement:** Prevent IDOR (Insecure Direct Object Reference) to ensure users can only modify their own profiles.
- **Activity Logging:** Record all profile modifications to maintain a secure audit trail for administrative review.

3 Architecture & Components

3.1 Internal Modules

- **ProfileController:** Handles HTTP GET and PUT requests, extracting user IDs directly from the verified JWT.
- **ProfileService:** Executes business logic, determining which fields are editable based on the user's role.
- **UserRepository:** Interfaces with the database to fetch and update the user's profile records.
- **ValidationUtils:** A utility module that enforces regex constraints (e.g., valid phone numbers, address lengths).
- **AuditLogger:** Encrypts and logs the "before" and "after" states of the profile data.

4 Profile Management Lifecycle

State	Trigger	Description
View Requested	Client navigation	User accesses the "My Profile" page; system fetches data.
Edit Initiated	Form submission	User submits new contact/address details.
Validated	Backend check	Inputs are checked for data type, length, and malicious code.
Persisted	Database update	Validated data overwrites existing records in the database.
Audited	Post-update hook	The change is logged, and a success response is returned.

5 API Endpoints

Method	Endpoint	Description
GET	/api/user/profile	Fetch the profile data for the authenticated user.
PUT	/api/user/profile	Update mutable profile fields (contact, address).
GET	/api/user/{id}/profile	(Admin/Faculty Only) View another user's public profile.

6 Data Model

6.1 User Profile Entry (MySQL)

Field	Type	Description
user_id	UUID	Foreign key linking to the central Auth User table.
full_name	VARCHAR	User's legal name (Read-only for Students).
phone_number	VARCHAR	User's primary contact number.
local_address	TEXT	Current residential or hostel address.
emergency_contact	VARCHAR	Name and number of emergency contact person.
profile_picture	VARCHAR	URL/path to the user's uploaded avatar.

7 Security Considerations

- **IDOR Prevention:** The service ignores any `user_id` sent in the payload for PUT requests. It strictly uses the `user_id` extracted from the trusted JWT to apply updates.
- **Read-Only Fields:** Fields like `cgpa`, `institute_id`, and `department` are stripped from incoming requests at the controller level to prevent unauthorized data tampering.
- **Data Sanitization:** All string inputs are passed through a sanitizer to strip HTML tags and prevent Stored XSS attacks.

8 Inter-Service Integration

1. Client sends a GET request with a Bearer Token to view their profile.
2. API Gateway routes the request to ProfileController.
3. ProfileController extracts the JWT payload, isolating the `user_id`.
4. ProfileService requests the data from UserRepository and formats the response.
5. Client submits a PUT request with updated data; ValidationUtils checks the payload before saving.

9 Error Handling

Error Code	HTTP	Description
VALIDATION_FAILED	400	The provided phone number or address format is invalid.
UNAUTHORIZED_UPDATE	403	User attempted to modify a restricted, read-only field.
USER_NOT_FOUND	404	The requested profile record does not exist in the database.
PAYLOAD_TOO_LARGE	413	The uploaded profile picture exceeds the size limit.

10 Configuration Parameters

Parameter	Default	Description
MAX_ADDRESS_LENGTH	255	Maximum allowed characters for the address field.
ALLOWED_IMAGE_TYPES	png, jpg	Permitted file formats for profile avatars.
MAX_AVATAR_SIZE_MB	2	Maximum file size allowed for image uploads.

11 Sequence Diagram Observations

(Diagrams maintained externally)

- **Read Profile Flow:** A straightforward read operation. The Controller extracts the ID from the JWT, the Service fetches it from the Database, and returns it. High traffic endpoints like this may utilize Redis caching to reduce database load.
- **Update Profile Flow:** The critical path involves the `ValidationUtils`. Before the `UserRepository` is allowed to execute an UPDATE statement, the input is strictly validated. Upon a successful update, the `AuditLogger` asynchronously records what fields were changed (e.g., "Updated phone_number from X to Y").

12 Class Diagram Responsibilities

(Diagrams maintained externally)

- **ProfileController:** Handles incoming profile requests, enforcing the extraction of the `userId` from the authenticated token to prevent IDOR.

- **ProfileService:** Contains the business logic for filtering read-only fields out of update requests and structuring the response payloads.
- **UserProfile (Entity):** Represents the data structure for the user's demographic and contact information.
- **ValidationUtils:** Contains static methods and regular expressions to validate phone numbers, emails, and text limits.
- **UserRepository:** Manages the SQL queries (SELECT and UPDATE) for the profile tables.
- **AuditLogger:** Dedicated service to log data mutations securely.

Academic Section & Admin Management System

Activity Logging & Auditing Service

Service Documentation — System Design

1 Service Overview

The Activity Logging & Auditing Service is a foundational security component responsible for the immutable tracking of all critical system events. It captures user access patterns, sensitive data modifications (e.g., grade updates, hostel leave approvals), and authentication anomalies. To comply with data privacy standards, all sensitive payloads within the logs are encrypted at rest, ensuring that even direct database access does not expose sensitive operational data without the decryption keys.

2 Core Responsibilities

- **Event Ingestion:** Asynchronously receive and queue log events from all other microservices (Auth, Academic, Hostel).
- **Payload Encryption:** Encrypt sensitive "before" and "after" data values using AES-256 before writing to the database.
- **Tamper-Evidence:** Generate cryptographic hashes for log entries to detect unauthorized modification or deletion of audit trails.
- **Data Persistence:** Store high-volume log data efficiently in an append-only relational or NoSQL database.
- **Audit Querying:** Provide a secure, paginated API for Super Admins to search, filter, and export system activity reports.

3 Architecture & Components

3.1 Internal Modules

- **AuditController:** Exposes internal endpoints for microservices to push logs, and external endpoints for Admins to view logs.
- **LogService:** Orchestrates the formatting, encryption, and hashing of incoming event streams.
- **CryptoManager:** Manages symmetric encryption keys and algorithms for securing log payloads.
- **LogRepository:** Handles database interactions using strict, append-only database credentials.
- **ArchivalWorker:** A background cron job that archives logs older than the retention period to cold storage.

4 Logging Lifecycle

State	Trigger	Description
Triggered	Service Action	A user action (e.g., "Upload Grades") triggers a log event.
Intercepted	Middleware	The application passes the event data to the Audit SDK/Controller.
Encrypted	Processing	Sensitive payload data is encrypted via CryptoManager.
Persisted	DB Write	The encrypted log is appended to the audit_logs table.
Archived	TTL Expiration	Logs older than 90 days are moved to secure cold storage.

5 API Endpoints

Method	Endpoint	Description
POST	/api/logs/event	(Internal) Ingest a new activity log from a microservice.
GET	/api/logs/audit	(Admin Only) Retrieve paginated, decrypted audit logs.
GET	/api/logs/export	(Admin Only) Export a filtered audit trail as a CSV/PDF.

6 Data Model

6.1 Audit Log Entry (PostgreSQL / MySQL)

Field	Type	Description
log_id	UUID	Primary key for the log entry.
user_id	UUID	ID of the user who performed the action.
action	VARCHAR	Standardized action name (e.g., `GRADE_UPDATE`, `LOGIN`).
resource	VARCHAR	The system module affected (e.g., `ACADEMIC_ENGINE`).
encrypted_data	TEXT	AES-256 encrypted JSON containing "old" and "new" values.
ip_address	VARCHAR	Originating IP address of the request.
created_at	TIMESTAMP	Exact time the event occurred.

7 Security Considerations

- **Append-Only Database User:** The database credentials used by the LogService only have `INSERT` and `SELECT` privileges. `UPDATE` and `DELETE` are strictly revoked at the database level.
- **Asynchronous Processing:** To prevent logging from slowing down critical operations (like Course Registration), logs are ingested asynchronously using a fire-and-forget mechanism or message queue.
- **Separation of Duties:** Keys used to encrypt the logs are stored in a secure Key Management System (KMS) separate from the database.

8 Inter-Service Integration

1. A faculty member updates a student's grade in the Academic Module.
2. The Academic Controller emits an event: {`action: "GRADE_UPDATE", user: "fac_123", old: "B", new: "A"`}.
3. The Activity Logging Service receives the event asynchronously.
4. The `old` and `new` values are encrypted into a single string.
5. The log is saved to the database without blocking the faculty member's HTTP response.

9 Error Handling

Error Code	HTTP	Description
INGESTION_FAILED	500	Database timeout; log is pushed to a retry queue.
UNAUTHORIZED_ACCESS	403	Non-admin attempted to query the audit logs.
DECRYPTION_ERROR	500	KMS failure or invalid key during log retrieval.
INVALID_PAYLOAD	400	The incoming log event is missing required tracking fields.

10 Configuration Parameters

Parameter	Default	Description
LOG_RETENTION_DAYS	90	Days to keep logs in the hot database before archival.
ENCRYPTION_ALGORITHM	AES-256-GCM	Cipher used for payload encryption.
ENABLE_ASYNC_QUEUE	true	Use an event queue instead of blocking HTTP calls.

11 Sequence Diagram Observations

(Diagrams maintained externally)

- **Log Ingestion Flow:** This sequence highlights the asynchronous nature of the service. The originating controller (e.g., Course Manager) fires the log event and immediately continues its execution. The Logger encrypts the payload and commits it to the database independently.
- **Audit Retrieval Flow:** When an Admin requests an audit report, the Logger retrieves the encrypted rows from the database. It then uses the `CryptoManager` to decrypt the `encrypted_data` field on-the-fly before formatting and returning the JSON/CSV response to the Admin frontend.

12 Class Diagram Responsibilities

(Diagrams maintained externally)

- **AuditController:** Handles HTTP/RPC endpoints for both ingestion (internal) and querying (external).

- **LogService:** The central orchestrator that ensures every log has a timestamp, formats the payload, and manages the encryption/decryption pipeline.
- **CryptoManager:** Abstract utility handling the AES-256-GCM encryption and decryption of JSON payloads using secure environment keys.
- **LogRepository:** Executes strict `INSERT` and `SELECT` queries. Ensures no updates or deletions are allowed.
- **LogEntry (Entity):** Represents the exact schema of the audit log stored in the database.

Academic Section & Admin Management System

Session Security & Persistent JWT Handling Service

Varshith Grandhi
Roll No: 230101041

Service Documentation — System Design

1 Service Overview

The Session Security Service manages the lifecycle, validation, and secure storage of user sessions across the application. Since the system uses stateless JSON Web Tokens (JWT) for authentication, this service is responsible for securely delivering these tokens (via HttpOnly cookies), validating them on every incoming request, refreshing them to maintain persistent user states, and explicitly revoking them (via Redis blocklisting) during logouts or security events.

2 Core Responsibilities

- **Token Validation:** Verify the cryptographic signature and expiration of JWTs on every protected API request.
- **Session Persistence:** Implement token refresh mechanisms (sliding sessions) to keep active users logged in without requiring constant re-authentication.
- **Token Revocation:** Maintain a high-speed Redis blocklist to invalidate compromised tokens or handle explicit user logouts.
- **Cookie Management:** Securely attach and clear HTTP-Only, Secure, and SameSite cookies to prevent client-side token theft.
- **CSRF Prevention:** Ensure that token delivery and validation mechanisms mitigate Cross-Site Request Forgery attacks.

3 Architecture & Components

3.1 Internal Modules

- **SessionMiddleware:** Intercepts requests at the API Gateway to validate cookies/tokens before routing to microservices.
- **TokenManager:** Handles the decoding, verifying, and issuing of Access and Refresh tokens.
- **RedisBlocklist:** A high-performance, in-memory cache used to store the IDs (jti) of revoked tokens until they naturally expire.
- **CookieHandler:** Utility for formatting secure HTTP response headers for token storage and deletion.
- **SessionController:** Exposes endpoints for token refreshing and session termination.

4 Session Lifecycle

State	Trigger	Description
Issued	Login Success	Access and Refresh tokens are generated and sent via cookies.
Validated	API Request	Middleware verifies token signature and checks Redis blocklist.
Refreshed	Token Expiring	Client uses Refresh token to securely obtain a new Access token.
Revoked	User Logout	Token ID is added to the Redis blocklist, cookies are cleared.
Expired	TTL Elapsed	Token naturally expires; user must re-authenticate.

5 API Endpoints

Method	Endpoint	Description
POST	/api/session/refresh	Validates refresh token and issues a new access token.
POST	/api/session/logout	Adds current token to blocklist and clears client cookies.
GET	/api/session/status	Checks if the current session is active (for UI rendering).
POST	/api/session/revoke-all	(Admin/Security) Revokes all active sessions for a specific user.

6 Data Model

6.1 Redis Blocklist Entry (Key-Value Store)

Field	Type	Description
Key	String	Formatted as `bl_token:{JWT_ID (jti)}`.
Value	String	User ID associated with the revoked token.
TTL	Integer	Time-To-Live matches the exact remaining lifespan of the JWT.

7 Security Considerations

- **XSS Mitigation:** Tokens are never stored in `localStorage` or `sessionStorage`. They are strictly delivered via `HttpOnly` cookies, making them inaccessible to malicious JavaScript.
- **CSRF Mitigation:** Cookies are configured with `SameSite=Strict` (or `Lax` depending on cross-origin needs) to prevent forged requests from external domains.
- **Stateless Revocation:** Since JWTs cannot be "deleted" from the server, the Redis Blocklist acts as an ultra-fast lookup to reject tokens that are structurally valid but logically revoked.
- **Sliding Expiration:** Short-lived Access Tokens (e.g., 15 mins) limit the window of opportunity for stolen tokens, while Refresh Tokens (e.g., 7 days) maintain user convenience.

8 Inter-Service Integration

1. Client browser sends an API request (e.g., `GET /grades`) automatically including the HttpOnly cookie.
2. SessionMiddleware reads the cookie and passes the token to TokenManager.
3. TokenManager verifies the signature algorithm and expiration date.
4. RedisBlocklist is queried. If the token's `jti` exists in Redis, request is rejected.
5. If valid, the user payload is attached to the request object (`req.user`) and passed downstream.

9 Error Handling

Error Code	HTTP	Description
TOKEN_EXPIRED	401	The Access Token has naturally expired (triggers refresh flow).
TOKEN_INVALID	401	The token is malformed or the signature does not match.
SESSION_REVOKED	401	The token is structurally valid but exists in the Redis blocklist.
COOKIE_MISSING	401	No session cookie was found in the incoming request headers.

10 Configuration Parameters

Parameter	Default	Description
ACCESS_TOKEN_TTL	15m	Lifespan of the primary API access token.
REFRESH_TOKEN_TTL	7d	Lifespan of the persistent background refresh token.
COOKIE_SECURE	true	Ensures cookies are only sent over HTTPS.
REDIS_URI	redis://local	Connection string for the blocklist cache server.

11 Sequence Diagram Observations

(Diagrams maintained externally)

- **Persistent Session Flow:** On page refresh or new app initialization, the frontend silently hits the `/session/status` or `/session/refresh` endpoint. The backend validates the HttpOnly cookie and either restores the application state or forces a redirect to the login screen, ensuring smooth UX without sacrificing security.
- **Revocation Flow:** During logout, the backend doesn't just clear the cookie (which only affects the client). It actively extracts the token ID (`jti`) and pushes it to Redis with an expiration time equal to the token's remaining lifespan. This guarantees that even if the token was copied elsewhere, it becomes useless instantly.

12 Class Diagram Responsibilities

(Diagrams maintained externally)

- **SessionMiddleware:** Intercepts HTTP requests, extracts cookies, and orchestrates validation before passing control to routers.
- **TokenManager:** Utility class managing cryptographic operations (signing, verifying, decoding) for JSON Web Tokens.
- **RedisBlocklist:** Encapsulates Redis `SETEX` and `EXISTS` commands to manage revoked tokens at extremely low latency.
- **CookieHandler:** Standardizes how cookies are serialized, setting `Secure`, `HttpOnly`, and `SameSite` flags consistently.
- **SessionController:** Handles HTTP requests specifically aimed at modifying the session state (refreshing or logging out).

Academic Section & Admin Management System

Student Dashboard & Data Aggregation Service

Service Documentation — System Design

1 Service Overview

The Student Dashboard Service operates using the Backend-For-Frontend (BFF) pattern. Instead of the React frontend making multiple separate API calls to various microservices (Academic, Hostel, Admin), this service acts as a single aggregation gateway. It concurrently fetches a student's enrolled courses, recent grades, upcoming assignment deadlines, hostel leave status, and global announcements, combining them into a single, optimized JSON payload to render the central student portal efficiently.

2 Core Responsibilities

- **Data Aggregation:** Execute parallel internal requests to downstream microservices to gather comprehensive student data.
- **Response Formatting:** Restructure and filter raw downstream data into a clean, UI-ready payload.
- **Caching:** Utilize Redis to cache relatively static dashboard data (like global announcements) to reduce downstream database load.
- **Partial Failure Handling:** Ensure the dashboard still loads (with degraded functionality) even if a non-critical downstream service (e.g., Hostel module) is temporarily unavailable.
- **Alerts Summarization:** Highlight critical actionable items, such as unpaid mess fees or impending assignment deadlines.

3 Architecture & Components

3.1 Internal Modules

- **DashboardController:** The single entry point for the frontend to request dashboard data.
- **AggregationService:** Orchestrates asynchronous, parallel calls to downstream services using `Promise.all()`.
- **AcademicClient:** Internal HTTP/gRPC client to fetch course, grade, and attendance data.
- **HostelClient:** Internal client to fetch active leave requests and mess subscription status.
- **AdminClient:** Internal client to fetch global announcements and alerts.
- **CacheManager:** Manages Redis read/write operations for high-traffic, low-mutation data.

4 Dashboard Aggregation Lifecycle

State	Trigger	Description
Requested	Page Load	Client requests the `/dashboard` endpoint with a valid JWT.
Cache Check	Pre-fetch	System checks Redis for cached global announcements.
Scatter	Parallel Fetch	Service simultaneously queries Academic, Hostel, and Admin APIs.
Gather	Promise Resolve	Responses are collected; timeouts or errors are handled gracefully.
Assembled	Serialization	Data is merged into a single structured JSON object and returned.

5 API Endpoints

Method	Endpoint	Description
GET	/api/student/dashboard	Main aggregated payload for the student home page.
GET	/api/student/dashboard/alerts	Lightweight polling endpoint for unread notifications/deadlines.
POST	/api/student/dashboard/refresh	Force bypass cache to retrieve the absolute latest data.

6 Data Model

6.1 Aggregated Dashboard Payload (JSON / Redis Cache Object)

Note: As an aggregation service, this represents the structural response rather than a direct SQL table.

Field	Type	Description
student_summary	Object	Basic profile info (Name, Roll No, Semester).
active_courses	Array	List of currently registered courses and attendance %.
pending_assignments	Array	Deadlines for upcoming academic submissions.
hostel_status	Object	Current mess plan and status of any pending leave requests.
announcements	Array	High-priority global broadcasts from the Admin portal.

7 Security Considerations

- **Token Forwarding:** The Dashboard Service does not have elevated privileges. It securely forwards the user's incoming JWT to downstream services so RBAC is enforced at the data-source level.
- **Data Leakage Prevention:** The AggregationService strips out internal database IDs and backend metadata from downstream responses before sending the final payload to the frontend.

- **Rate Limiting:** To prevent DDoS attacks from exhausting internal microservices, the `/dashboard` endpoint is strictly rate-limited per user.

8 Inter-Service Integration

1. The React frontend sends a GET request to `/api/student/dashboard`.
2. The Dashboard Aggregator initiates three parallel HTTP requests to the Academic, Hostel, and Admin microservices.
3. The Aggregator waits for a maximum of 2000ms (Timeout limit).
4. If the Hostel service times out, the Aggregator catches the error, sets `hostel_status: null`, but still successfully returns the Academic and Admin data.
5. The frontend renders the available data and shows a localized error state for the Hostel widget.

9 Error Handling

Error Code	HTTP	Description
PARTIAL_CONTENT	206	Returned if one or more downstream services fail or timeout.
GATEWAY_TIMEOUT	504	All downstream services failed to respond within the threshold.
UNAUTHORIZED	401	The provided student JWT is invalid or expired.
SERVICE_UNAVAILABLE	503	The aggregator itself is down or overloaded.

10 Configuration Parameters

Parameter	Default	Description
DOWNSTREAM_TIMEOUT_MS	2000	Max time to wait for a microservice before returning a partial response.
ANNOUNCEMENT_CACHE_TTL	300	Seconds to cache admin announcements in Redis.
MAX_CONCURRENT_REQUESTS	1000	Circuit breaker limit to prevent internal network flooding.

11 Sequence Diagram Observations

(Diagrams maintained externally)

- **Parallel Fetching:** The sequence diagram heavily features a `par` (parallel) block. This is critical for performance; if Academic data takes 500ms and Hostel data takes 400ms, parallel fetching ensures the total wait time is 500ms, rather than 900ms (sequential).

- **Gateway Role:** The diagram illustrates how the Frontend only communicates with the single Node.js Gateway. The Gateway abstracts the complexity of navigating the underlying microservice databases (Academic Engine DB vs. Admin Comm DB).

12 Class Diagram Responsibilities

(Diagrams maintained externally)

- **DashboardController:** Handles the HTTP request/response cycle, enforcing JWT presence.
- **AggregationService:** The core business logic layer. Implements `Promise.all()` to concurrently execute client requests and handles the assembly and formatting of the final JSON tree.
- **AcademicClient / AdminClient / HostelClient:** Specialized HTTP/REST wrappers. They manage the exact URL routing, headers, and error parsing for their respective downstream microservices.
- **CacheManager:** Abstract interface interacting with Redis to retrieve or set data that doesn't need to be fetched dynamically on every page load.

Academic Section & Admin Management System

Course Registration Service

Service Documentation — System Design

1 Service Overview

The Course Registration Service is a core component of the Student Academic Portal, providing a self-service module for students to enroll in courses for the current semester. It acts as a high-concurrency engine that validates student eligibility, checks course prerequisites, manages seat capacity in real-time, and ensures that students adhere to credit limit policies. This service transitions the registration process from a manual, paper-based workflow to a centralized digital system.

2 Core Responsibilities

- **Enrollment Processing:** Manage the formal registration of students into specific course codes.
- **Prerequisite Verification:** Automatically check if a student has completed the required prior coursework before allowing registration.
- **Capacity Management:** Monitor and enforce maximum enrollment limits for each course section to prevent over-allocation.
- **Schedule Conflict Detection:** Ensure that a student does not register for two courses with overlapping time slots.
- **Credit Limit Enforcement:** Validate that the total credits selected fall within the minimum and maximum allowable range for the semester.

3 Architecture & Components

3.1 Internal Modules

- **RegistrationController:** Handles student requests to view available courses and submit registration forms.
- **RegistrationService:** Orchestrates the business logic, including the sequential validation of prerequisites and capacity.
- **PrereqChecker:** A specialized module that queries historical academic records to verify eligibility.
- **CapacityManager:** Manages atomic increments and decrements of seat counts to ensure data consistency during concurrent registrations.
- **EnrollmentRepository:** Executes database transactions to record successful course-student mappings.

4 Registration Lifecycle

State	Trigger	Description
Browsing	Course Catalog View	Student views list of available courses and remaining seats.
Selecting	Add to Cart	Student selects a course for validation.
Validating	Form Submission	System checks prerequisites, capacity, and schedule conflicts.
Committed	Success	The enrollment is saved to the database; seat count is decremented.
Confirmed	Notification	Student receives a registration snapshot on their dashboard.

5 API Endpoints

Method	Endpoint	Description
GET	/api/courses/available	Fetch list of courses open for registration in current semester.
POST	/api/courses/register	Submit a registration request for one or more course IDs.
GET	/api/courses/my-enrollments	View a list of courses the student is currently registered for.

6 Data Model

6.1 Enrollment Record (MySQL)

Field	Type	Description
enrollment_id	UUID	Primary key for the enrollment record.
student_id	UUID	Reference to the Student table.
course_id	UUID	Reference to the Course Catalog table.
semester_code	VARCHAR	Current semester identifier (e.g., "SPRING_2024").
status	ENUM	Current status: 'REGISTERED', 'PENDING', 'DROPPED'.
registration_date	TIMESTAMP	Record of when the student enrolled.

7 Security Considerations

- **Concurrency Control:** Uses database transactions and row-level locking (SELECT FOR UPDATE) to ensure that seat capacity is never exceeded during simultaneous registration attempts.
- **Registration Window Enforcement:** Strict server-side checks to ensure registration only occurs during the officially allocated time window.

- **Role Authorization:** Only users with the 'STUDENT' role can access the registration endpoints.
- **Input Validation:** Sanitization of course IDs to prevent SQL injection.

8 Inter-Service Integration

1. Registration Service fetches the list of available courses from the **Admin Course Catalog**.
2. It queries the **Academic Engine** to verify the student's historical grades for prerequisite checks.
3. Upon successful registration, it triggers the **Activity Logger** to record the enrollment.
4. It sends a notification to the **Communication System** to update the student dashboard.

9 Error Handling

Error Code	HTTP	Description
PREREQ_NOT_MET	403	Student has not completed the required introductory courses.
COURSE_FULL	409	The course has reached its maximum seat capacity.
SCHEDULE_CONFLICT	400	The selected course overlaps with another registered course.
CREDIT_LIMIT_EXCEEDED	422	Total credits exceed the maximum allowed for the semester.

10 Configuration Parameters

Parameter	Default	Description
MAX_CREDITS	24	Maximum number of credits a student can take per semester.
MIN_CREDITS	12	Minimum number of credits required for full-time status.
REGISTRATION_OPEN	false	Global flag to enable/disable the registration module.
DEFAULT_SEAT_CAPACITY	60	Default limit if not specified for a course section.

11 Sequence Diagram Observations

(Diagrams maintained externally)

- **Registration Flow:** The sequence illustrates the student selecting a course, the system checking the database for prerequisite completion, verifying that seats are available, and finally performing an atomic update on the Enrollment and Course tables.

- **Validation Failure:** If any check fails (e.g., prerequisite not found), the sequence shows an immediate exit with a 403/409 error, ensuring the database state remains unchanged and the student is notified of the specific reason for failure.

12 Class Diagram Responsibilities

(Diagrams maintained externally)

- **RegistrationController:** Acts as the API entry point, validating the session and parsing the course IDs.
- **RegistrationService:** The main logic hub that sequences the prerequisite check, capacity check, and conflict check.
- **PrereqChecker:** Encapsulates the logic for traversing the student's academic history.
- **CapacityManager:** Handles the logic for managing and locking seat counts to ensure data integrity.
- **EnrollmentRepository:** Provides the data access layer for creating and fetching enrollment records.

Academic Section & Admin Management System

Dropped Courses Service

Service Documentation — System Design

1 Service Overview

The Dropped Courses Service manages the formal withdrawal of students from specific academic courses during the officially sanctioned "Add/Drop" window. This service ensures that students can modify their academic load while adhering to institutional deadlines and minimum credit requirements. It acts as a critical sub-module of the Academic Engine, synchronizing with the Program Configuration module to enforce registration timelines [cite: 128, 151].

2 Core Responsibilities

- **Deadline Enforcement:** Verify that the course drop request is submitted within the allocated registration window.
- **Enrollment Verification:** Ensure the student is currently registered for the course they are attempting to drop.
- **Status Management:** Update the enrollment record status from 'ACTIVE' to 'DROPPED' in the academic database.
- **Credit Validation:** (Optional) Check if dropping the course violates minimum credit requirements for the semester.
- **Notification Trigger:** Alert relevant faculty members of the change in their course enrollment list.
- **Audit Trail:** Maintain a secure log of the drop action, including timestamps and user identity.

3 Architecture & Components

3.1 Internal Modules

- **RegistrationController:** Handles HTTP POST requests for course withdrawal.
- **DropService:** Contains the core logic for validating deadlines and academic rules.
- **ProgramConfigManager:** Fetches active semester timelines and drop-window deadlines.
- **AcademicRepository:** Executes the database updates on the student enrollment tables.
- **AuditLogger:** Logs the transaction details for administrative review and troubleshooting.

4 Course Drop Lifecycle

State	Trigger	Description
Initiated	Student Action	Student selects a course to drop from their active dashboard.
Window-Checked	System Validation	Service verifies the current timestamp against the Add/Drop window.
Validated	Eligibility Check	System confirms the student is enrolled and meets drop criteria.
Dropped	DB Transaction	Enrollment status is updated to 'DROPPED'; seat count is released.
Finalized	Logged	System records the drop event and sends a confirmation to the user.

5 API Endpoints

Method	Endpoint	Description
POST	/api/courses/drop	Process a request to drop a course (requires Course ID).
GET	/api/courses/drop-status	Check if the drop window is currently active for the user.
GET	/api/student/dropped-history	Retrieve a list of all courses dropped by the student.

6 Data Model

6.1 Enrollment Update Schema (MySQL)

Field	Type	Description
enrollment_id	UUID	Primary key linking the student to the specific course.
status	ENUM	Transitions from 'ACTIVE' to 'DROPPED'.
dropped_at	TIMESTAMP	The exact time the drop request was processed.
semester_id	VARCHAR	The active academic term (e.g., 2024-Autumn).
reason_code	VARCHAR	Optional code provided by the student for dropping.

7 Security Considerations

- **Identity-Based Drop:** The system uses the 'student_id' embedded in the JWT. A student cannot drop a course for another student by tampering with request IDs.
- **Temporal Security:** The Drop window is checked on the server-side, preventing "time-travel" attacks where users try to modify their local system clock to bypass deadlines.
- **Integrity:** The drop operation is performed within a database transaction to ensure that if the log fails, the status change is rolled back.

8 Inter-Service Integration

1. Student clicks "Drop" on the Student Academic Portal.
2. **DropService** calls **ProgramConfigModule** to verify the active drop window.
3. **DropService** calls **AcademicRepository** to fetch current enrollment.
4. Upon success, **AcademicRepository** updates the record status.
5. **AuditLogger** records the transaction, and the student's dashboard is refreshed.

9 Error Handling

Error Code	HTTP	Description
WINDOW_CLOSED	403	The official deadline for dropping courses has passed.
NOT_ENROLLED	404	Student is not registered for the course ID provided.
MIN_CREDIT_VIOLATION	422	Dropping this course puts the student below the min-credit threshold.
ALREADY_DROPPED	400	The course has already been dropped previously.

10 Configuration Parameters

Parameter	Default	Description
DROP_WINDOW_DAYS	14	Days from semester start that the drop window remains open.
MIN_SEM_CREDITS	12	Minimum credits a student must maintain after a drop.
NOTIFY_FACULTY_ON_DROP	true	Flag to send automated emails to course instructors.

11 Sequence Diagram Observations

(Diagrams maintained externally)

- **Drop Process Flow:** This flow highlights the interaction with the `ProgramConfigManager`. Before any database modification occurs, the system must receive a "True" status from the configuration manager regarding the window's validity.
- **Validation Failure Flow:** If a student attempts to drop a core course that cannot be dropped, or if the deadline has passed, the `DropService` raises an exception, which is caught by the global error handler to return a clear, localized error message to the student UI.

12 Class Diagram Responsibilities

(Diagrams maintained externally)

- **RegistrationController:** Endpoint for the drop functionality; ensures the JWT is valid and the request payload is well-formed.

- **DropService:** The orchestrator of the drop business rules, coordinating between configuration checks and database updates.
- **ProgramConfigManager:** Central authority on academic dates, registration periods, and deadlines.
- **AcademicRepository:** Direct interface for MySQL operations; handles the specific UPDATE queries for the enrollment table.
- **AuditLogger:** Service responsible for creating an immutable record of academic changes.

Academic Section & Admin Management System

Academic Record Viewer Service

Service Documentation — System Design

1 Service Overview

The Academic Record Viewer Service provides students with a secure, real-time interface to access their educational progress. It aggregates data from the Academic Engine to display current semester marks, historical letter grades, and calculated performance metrics such as Semester Grade Point Average (SGPA) and Cumulative Grade Point Average (CGPA). This service acts as the primary "Read-Only" portal for students to monitor their academic standing without needing to visit the Academic Section office.

2 Core Responsibilities

- **Data Aggregation:** Fetch historical and current course marks from the central enrollment and grading databases.
- **GPA Calculation:** Dynamically compute SGPA and CGPA based on credit weights and letter grade points.
- **Historical Tracking:** Provide a chronological view of all completed semesters and corresponding performance.
- **Privacy Enforcement:** Ensure a student can only access their own records via strict JWT-based ownership checks.
- **Audit Visibility:** Display faculty feedback and qualitative comments alongside numerical marks where applicable.

3 Architecture & Components

3.1 Internal Modules

- **RecordController:** Intercepts student requests and extracts the `student_id` from the session token.
- **RecordService:** Orchestrates the retrieval of raw marks and coordinates with the GPA Engine for processing.
- **GPAEngine:** A logic module containing the institutional formulas for grade-to-point conversion and weighted averages.
- **GradeRepository:** Executes optimized SQL queries to join student records with course credit information.
- **CacheManager:** Temporarily stores calculated CGPA values in Redis to improve dashboard load times.

4 Academic Record Lifecycle

State	Trigger	Description
Requested	Dashboard Access	User navigates to the "My Grades" section.
Retrieved	DB Query	Raw grade data is fetched for the specific Student ID.
Computed	GPA Processing	The GPAEngine calculates averages using active credit rules.
Formatted	UI Mapping	Data is structured into a per-semester tabular format.
Displayed	Frontend Render	The final report is rendered on the student's portal.

5 API Endpoints

Method	Endpoint	Description
GET	/api/student/grades	Fetch current semester marks and active feedback.
GET	/api/student/history	Retrieve all past semester grades and final CGPA.
GET	/api/student/summary	Returns a high-level summary for dashboard widgets.

6 Data Model

6.1 Grade Record Schema (MySQL)

Field	Type	Description
student_id	UUID	Reference to the Student.
course_code	VARCHAR	Unique identifier for the subject (e.g., CS345).
letter_grade	CHAR(2)	The assigned grade (e.g., A, B+, F).
grade_points	INT	Numeric value associated with the grade.
semester	VARCHAR	Academic term (e.g., Autumn 2023).
is_published	BOOLEAN	Flag to hide grades until officially released by Admin.

7 Security Considerations

- **Ownership Validation:** The service ignores any user ID passed in parameters and strictly uses the ID found in the cryptographically signed JWT.
- **Publishing Logic:** Students cannot view grades for the current semester until the "is_published" flag is toggled by the Academic Section Staff.
- **Data Integrity:** The Record Viewer is strictly "Read-Only." No PUT or POST methods are exposed that could modify grade data.
- **Rate Limiting:** To prevent scraping of academic data, limits are placed on how frequently the full history can be refreshed.

8 Inter-Service Integration

1. Student requests their Academic Record via the React Portal.

2. The Record Viewer Service calls the `CourseCatalogControl` to get credit weights for each course.
3. The service fetches raw grades from the `GradeRepository`.
4. Data is passed to the `GPAEngine` to calculate a real-time CGPA.
5. The unified payload is returned to the student portal.

9 Error Handling

Error Code	HTTP	Description
RECORDS_NOT_FOUND	404	No academic history exists for the given student ID.
GRADES_NOT_PUBLISHED	204	Grades exist but are currently hidden by administration.
CALCULATION_ERROR	500	Missing course credit data prevents GPA calculation.
UNAUTHORIZED_ACCESS	403	Attempt to view a record belonging to a different student ID.

10 Configuration Parameters

Parameter	Default	Description
GPA_SCALE	10.0	The maximum possible grade point value.
ROUNDING_PRECISION	2	Decimal places for SGPA/CGPA (e.g., 8.54).
ENABLE_LIVE_GPA	true	Calculate GPA on-the-fly vs. using pre-calculated DB values.
MIN_PASS_GRADE	D	The lowest grade that counts toward earned credits.

11 Sequence Diagram Observations

(Diagrams maintained externally)

- **Retrieval Flow:** The sequence shows the service performing a multi-table join. It combines user-specific grades with the institutional course catalog to ensure that GPA is calculated based on the correct credit weightages for that specific academic year.
- **Ownership Check:** The diagram explicitly highlights the middleware step where the JWT is decoded and the `student_id` is verified against the database request to block horizontal privilege escalation.

12 Class Diagram Responsibilities

(Diagrams maintained externally)

- **RecordController:** Entry point for fetching grade data; handles HTTP routing and error response formatting.
- **RecordService:** The business logic hub; aggregates grades and triggers calculations.

- **GPAEngine:** Logic-heavy class dedicated to the mathematical conversion of grades to weighted averages.
- **GradeRepository:** Encapsulates the SQL logic for the `grades` and `courses` tables.
- **StudentGrade (Entity):** A data object representing a single row of a student's performance in a subject.

Academic Section & Admin Management System

Attendance Monitoring Service

Service Documentation — System Design

1 Service Overview

The Attendance Monitoring Service provides students with a centralized and real-time mechanism to track their academic attendance across all enrolled courses. It enables users to view detailed day-wise attendance records and compute overall attendance percentage.

2 Core Responsibilities

- Daily attendance retrieval
- Percentage calculation
- Course-wise filtering
- Secure access control

3 Architecture & Components

3.1 Internal Modules

- AttendanceController
- AttendanceService
- AttendanceRepository
- CourseRepository

4 Service Lifecycle

State	Trigger	Description
Requested	Student Login	Student requests attendance data after authentication
Validated	Token Verified	JWT token is validated and user identity is confirmed
Fetched	API Call	Attendance records are retrieved from the database
Processed	Computation	Attendance percentage is calculated based on records
Displayed	UI Render	Data is displayed on the student dashboard

5 API Endpoints

Method	Endpoint	Description
GET	/api/attendance/daily?courseId=	Fetch day-wise attendance records
GET	/api/attendance/percentage?courseId=	Retrieve attendance percentage
GET	/api/attendance/summary	Fetch overall attendance summary

6 Data Model

6.1 Attendance Table

Field	Type	Description
attendanceId	INT (PK)	Unique attendance record identifier
studentId	INT (FK)	Reference to student entity
courseId	INT (FK)	Reference to course entity
date	DATE	Date of class session
status	ENUM	Attendance status (Present/Absent)

7 Security Considerations

- JWT-based authentication
- Role-based access control
- Secure HTTPS communication

8 Inter-Service Integration

1. Client sends request
2. Auth Service validates token
3. Attendance Service fetches data
4. Course Service validates enrollment
5. Response returned to client

9 Error Handling

Error Code	HTTP	Description
INVALID_USER	401	Unauthorized access attempt
DATA_NOT_FOUND	404	No attendance records available
COURSE_NOT_FOUND	404	Invalid course ID provided
SERVER_ERROR	500	Internal server error

10 Configuration Parameters

Parameter	Default	Description
CACHE_TIME	5 minutes	Cache duration for responses
MAX_QUERY_LIMIT	100	Maximum records per request
ATTENDANCE_THRESHOLD	75%	Minimum required attendance

11 Sequence Diagram Observations

The system follows a request-response flow where authenticated users request attendance, data is fetched, processed, and returned.

12 Class Diagram Responsibilities

- AttendanceController handles requests
- AttendanceService processes logic
- Repository interacts with database

Academic Section & Admin Management System

Assignment Upload Service

Service Documentation — System Design

1 Service Overview

The Assignment Upload Service provides a digital interface for students to submit assignment files securely within specified deadlines. The system ensures proper validation of submissions, prevents late uploads, and maintains a structured record of all submitted assignments.

This service integrates with assignment configuration and grading modules to form a complete assignment lifecycle.

2 Core Responsibilities

- Upload assignment files securely
- Enforce submission deadlines
- Prevent duplicate or invalid submissions
- Maintain submission records with timestamps

3 Architecture & Components

3.1 Internal Modules

- **AssignmentController:** Handles incoming upload requests and responses
- **AssignmentService:** Validates deadlines and processes submissions
- **SubmissionRepository:** Stores submission metadata in database
- **Storage Service:** Handles file storage (local/cloud)

4 Service Lifecycle

State	Trigger	Description
Requested	Upload Request	Student initiates assignment upload process
Validated	Deadline Check	System verifies whether submission is within allowed deadline
Processed	File Validation	File type and size are validated before storage
Stored	Save Operation	File is uploaded and metadata is stored in database
Completed	Response Sent	Confirmation is returned to the student

5 API Endpoints

Method	Endpoint	Description
POST	/api/assignment/upload	Upload assignment file with metadata
GET	/api/assignment/submission/status	Check submission status for a student
GET	/api/assignment/submission/list	Retrieve all submissions for a student

6 Data Model

6.1 Submission Table

Field	Type	Description
submissionId	INT (PK)	Unique identifier for submission
assignmentId	INT (FK)	Reference to assignment entity
studentId	INT (FK)	Reference to student entity
fileUrl	VARCHAR	Path or URL of uploaded file
submittedAt	DATETIME	Timestamp of submission
status	ENUM	Submitted / Late / Rejected

7 Security Considerations

- **Authentication:** Only logged-in students can upload files
- **File Validation:** Restrict file types (PDF, DOCX, ZIP)

- **Size Limitation:** Prevent oversized uploads
- **Secure Storage:** Store files in protected locations

8 Inter-Service Integration

1. Student sends upload request via frontend
2. Auth Service validates user identity
3. Assignment Service verifies deadline
4. Storage Service saves file
5. SubmissionRepository stores metadata
6. Response returned to client

9 Error Handling

Error Code	HTTP	Description
DEADLINE_PASSED	400	Submission attempted after deadline
INVALID_FILE	400	Unsupported file format
FILE_TOO_LARGE	413	Uploaded file exceeds allowed size
UNAUTHORIZED	401	User not authenticated
SERVER_ERROR	500	Internal system failure

10 Configuration Parameters

Parameter	Default	Description
MAX_FILE_SIZE	10MB	Maximum allowed upload size
ALLOWED_FILE_TYPES	PDF, DOCX, ZIP	Supported formats
SUBMISSION_DEADLINE_BUFFER	5 minutes	Grace period for submissions

11 Sequence Diagram Observations

The submission process follows a strict validation pipeline. The request is authenticated, deadline is verified, file is validated, and then stored. Only after successful completion, the submission is recorded in the database.

12 Class Diagram Responsibilities

- **AssignmentController:** Handles upload requests
- **AssignmentService:** Applies business rules and validation
- **SubmissionRepository:** Stores submission records
- **Submission Entity:** Represents uploaded assignments

Academic Section & Admin Management System

General Announcements & Group Communication Service

Service Documentation — System Design

1 Service Overview

The General Announcements and Group Communication Service provides a centralized platform for communication between faculty and students. Each course has a dedicated communication channel where announcements, updates, and discussions can take place.

This service ensures structured and course-specific communication, improving information dissemination and collaboration within academic environments.

2 Core Responsibilities

- Enable course-wise group communication channels
- Allow faculty to post announcements
- Allow students to participate in discussions
- Maintain message history for each course
- Ensure secure and authenticated communication

3 Architecture & Components

3.1 Internal Modules

- **CommunicationController:** Handles message sending and retrieval requests
- **CommunicationService:** Processes messages and manages business logic
- **MessageRepository:** Stores and retrieves messages from the database
- **CourseRepository:** Validates course membership
- **Notification Service:** Sends alerts for new messages or announcements

4 Service Lifecycle

State	Trigger	Description
Initiated	User Action	User opens course communication channel
Composed	Message Input	User writes message or announcement
Validated	Auth Check	User identity and course membership verified
Stored	Save Operation	Message saved in database
Broadcasted	Notification	Message delivered to all course participants
Retrieved	Fetch Request	Messages fetched and displayed to users

5 API Endpoints

Method	Endpoint	Description
POST	/api/message/send	Send a message to course channel
GET	/api/message/course?courseId=	Retrieve all messages for a course
GET	/api/message/announcements?courseId=	Fetch announcements only
DELETE	/api/message/delete?id=	Delete a message (authorized users only)

6 Data Model

6.1 Message Table

Field	Type	Description
messageId	INT (PK)	Unique identifier for message
senderId	INT (FK)	User who sent the message
courseId	INT (FK)	Course channel reference
messageText	TEXT	Content of message or announcement
messageType	ENUM	Normal / Announcement
sentAt	DATETIME	Timestamp of message

7 Security Considerations

- **Authentication:** Only logged-in users can send or view messages
- **Authorization:** Only course members can access messages

- **Role Control:** Only faculty can post announcements
- **Content Filtering:** Prevent malicious or inappropriate content

8 Inter-Service Integration

1. User sends message via frontend interface
2. Auth Service validates identity
3. Course Service verifies enrollment
4. Communication Service processes message
5. MessageRepository stores message
6. Notification Service alerts users

9 Error Handling

Error Code	HTTP	Description
UNAUTHORIZED	401	User not authenticated
ACCESS_DENIED	403	User not part of course
MESSAGE_EMPTY	400	Message content is empty
NOT_FOUND	404	Course or message not found
SERVER_ERROR	500	Internal system error

10 Configuration Parameters

Parameter	Default	Description
MAX_MESSAGE_LENGTH	2000 characters	Maximum allowed message size
NOTIFICATION_ENABLED	TRUE	Enable/disable notifications
MESSAGE_RETENTION_PERIOD	30 days	Duration for storing messages

11 Sequence Diagram Observations

The communication system follows a real-time interaction model. Messages are validated, stored, and broadcasted to all participants in the course channel. Retrieval operations fetch messages in chronological order.

12 Class Diagram Responsibilities

- **CommunicationController:** Handles API requests for messaging
- **CommunicationService:** Processes and manages message flow
- **MessageRepository:** Stores and retrieves messages
- **Message Entity:** Represents communication data

Academic Section & Admin Management System

Transcript Request Service

Service Documentation — System Design

1 Service Overview

The Transcript Request Service provides a digital platform for students to request official academic documents such as transcripts, bonafide certificates, and grade cards. The system enables seamless submission, tracking, and processing of requests by the administrative department.

This service improves efficiency by eliminating manual paperwork and providing transparent request tracking for students.

2 Core Responsibilities

- Allow students to submit transcript and document requests
- Provide real-time tracking of request status
- Enable administrative review and approval workflow
- Maintain records of all submitted requests
- Notify users about request updates

3 Architecture & Components

3.1 Internal Modules

- **TranscriptController:** Handles request submission and status queries
- **TranscriptService:** Manages business logic and workflow processing
- **TranscriptRepository:** Stores and retrieves request data
- **Admin Module:** Allows admin users to process and approve requests
- **Notification Service:** Sends updates to students regarding request status

4 Service Lifecycle

State	Trigger	Description
Requested	Submission	Student submits a new transcript request
Validated	Auth Check	User identity and request details are verified
Processing	Admin Review	Administrative staff reviews the request
Approved	Approval Action	Request is approved and processed
Rejected	Rejection Action	Request is rejected with reason
Completed	Finalization	Document is generated and delivered

5 API Endpoints

Method	Endpoint	Description
POST	/api/transcript/request	Submit a new transcript request
GET	/api/transcript/status?id=	Track status of a request
GET	/api/transcript/history	Retrieve all past requests for a student
PUT	/api/transcript/update-status	Admin updates request status

6 Data Model

6.1 Transcript Request Table

Field	Type	Description
requestId	INT (PK)	Unique identifier for request
studentId	INT (FK)	Reference to student entity
requestType	ENUM	Transcript / Bonafide / Grade Card
status	ENUM	Pending / Approved / Rejected / Completed
appliedAt	DATETIME	Timestamp when request was submitted
processedAt	DATETIME	Timestamp when request was processed
remarks	TEXT	Admin remarks or rejection reason

7 Security Considerations

- **Authentication:** Only authenticated users can submit and track requests

- **Authorization:** Only admin users can update request status
- **Data Privacy:** Sensitive academic records are securely handled
- **Audit Logging:** All actions are logged for accountability

8 Inter-Service Integration

1. Student submits request via frontend interface
2. Auth Service validates identity
3. Transcript Service records request
4. Admin Module processes request
5. Notification Service updates student
6. Final document delivered to student

9 Error Handling

Error Code	HTTP	Description
INVALID_REQUEST	400	Invalid request parameters
UNAUTHORIZED	401	User not authenticated
ACCESS_DENIED	403	Unauthorized action (non-admin)
NOT_FOUND	404	Request not found
SERVER_ERROR	500	Internal system error

10 Configuration Parameters

Parameter	Default	Description
PROCESSING_TIME	3 days	Average time to process requests
MAX_REQUESTS_PER_MONTH	5	Limit on number of requests per student
NOTIFICATION_ENABLED	TRUE	Enable notifications for updates

11 Sequence Diagram Observations

The service follows a structured workflow where the student initiates a request, the system validates it, and the administrative module processes it. Status updates are communicated back to the student at each stage.

12 Class Diagram Responsibilities

- **TranscriptController:** Handles API requests
- **TranscriptService:** Manages request lifecycle
- **TranscriptRepository:** Stores request data
- **Transcript Entity:** Represents request information

Academic Section & Admin Management System

Resource Sharing Service

Service Documentation — System Design

1 Service Overview

The Resource Sharing Service enables faculty members to upload and distribute academic materials such as lecture notes, presentations, assignments, and reference documents for each course. Students can access these resources in a structured, course-wise manner.

This service enhances the learning experience by providing centralized and easy access to all academic materials.

2 Core Responsibilities

- Allow faculty to upload course-specific resources
- Provide students with access to shared materials
- Organize resources by course and type
- Maintain metadata for each resource
- Ensure secure and role-based access

3 Architecture & Components

3.1 Internal Modules

- **ResourceController:** Handles upload and retrieval requests
- **ResourceService:** Manages resource processing and access control
- **ResourceRepository:** Stores and retrieves resource metadata
- **CourseRepository:** Validates course association
- **Storage Service:** Handles file storage and retrieval

4 Service Lifecycle

State	Trigger	Description
Initiated	Upload Request	Faculty initiates resource upload
Validated	Auth Check	Faculty identity and course association verified
Processed	File Validation	File type and size validated
Stored	Save Operation	File stored and metadata recorded
Published	Availability	Resource made accessible to students
Accessed	Fetch Request	Students retrieve and view/download resource

5 API Endpoints

Method	Endpoint	Description
POST	/api/resource/upload	Upload a new resource
GET	/api/resource/course?courseId=	Retrieve all resources for a course
GET	/api/resource/download?id=	Download specific resource file
DELETE	/api/resource/delete?id=	Remove resource (faculty only)

6 Data Model

6.1 Resource Table

Field	Type	Description
resourceId	INT (PK)	Unique identifier for resource
courseId	INT (FK)	Associated course
facultyId	INT (FK)	Faculty who uploaded resource
fileName	VARCHAR	Name of the uploaded file
fileUrl	VARCHAR	Storage location of file
resourceType	ENUM	Notes / Slides / Assignment / Other
uploadedAt	DATETIME	Timestamp of upload

7 Security Considerations

- **Authentication:** Only authorized users can access resources
- **Authorization:** Only faculty can upload or delete resources
- **Access Control:** Students can only view resources of enrolled courses
- **File Security:** Files stored securely with restricted access paths

8 Inter-Service Integration

1. Faculty uploads resource via frontend
2. Auth Service validates user role
3. Course Service verifies course association
4. Storage Service saves file
5. ResourceRepository stores metadata
6. Students access resources via API

9 Error Handling

Error Code	HTTP	Description
UNAUTHORIZED	401	User not authenticated
ACCESS_DENIED	403	User not permitted to perform action
INVALID_FILE	400	Unsupported file format
FILE_TOO_LARGE	413	File exceeds allowed size
NOT_FOUND	404	Resource not found
SERVER_ERROR	500	Internal system error

10 Configuration Parameters

Parameter	Default	Description
MAX_UPLOAD_SIZE	20MB	Maximum allowed file size
ALLOWED_FILE_TYPES	PDF, PPT, DOCX	Supported file formats
RESOURCE_RETENTION_PERIOD	365 days	Duration resources are stored

11 Sequence Diagram Observations

The service follows a structured upload and retrieval workflow. Faculty members upload resources, which are validated and stored. Students can then access these resources through course-specific endpoints.

12 Class Diagram Responsibilities

- **ResourceController:** Handles API requests for resources
- **ResourceService:** Manages processing and access control

- **ResourceRepository:** Stores and retrieves resource metadata
- **Resource Entity:** Represents shared resources

Academic Section & Admin Management System
Course Feedback Service
Service Documentation — System Design

1 Service Overview

The Course Feedback Service enables students to provide structured feedback on courses they are enrolled in. It ensures that only authorized students can submit feedback, securely stores feedback data, and provides administrative access for reviewing and analyzing feedback. The service plays a critical role in maintaining academic quality through continuous evaluation.

2 Core Responsibilities

- **Feedback Submission:** Allow students to submit ratings and comments for courses.
- **Validation:** Ensure that students can only provide feedback for courses they are enrolled in.
- **Data Storage:** Persist feedback data securely in the database.
- **Feedback Retrieval:** Provide course-wise feedback for analysis.
- **Administrative Notification:** Notify administrators about new feedback submissions.

3 Architecture & Components

3.1 Internal Modules

- **FeedbackController:** Entry point handling HTTP requests for feedback submission and retrieval.
- **FeedbackService:** Core business logic handler orchestrating validation, storage, and retrieval.
- **FeedbackRepository:** Handles persistence of feedback data.
- **CourseRepository:** Validates existence and legitimacy of courses.
- **AdminRepository:** Handles administrative notifications.

3.2 Entities

- **CourseFeedback:** Stores feedback details including student ID, course ID, comments, rating, and timestamp.
- **Student:** Represents student information.
- **Course:** Represents course metadata.
- **Admin:** Represents administrative users reviewing feedback.

4 Feedback Lifecycle

State	Trigger	Description
Requested	Feedback Form Opened	Student accesses feedback form.
Validated	Course Verification	System checks if student is enrolled in the course.
Submitted	Feedback Submission	Feedback is submitted with rating and comments.
Stored	Database Insert	Feedback is stored in the system database.
Reviewed	Admin Access	Admin reviews feedback through dashboard.

5 API Endpoints

Method	Endpoint	Description
GET	/eligible-courses	Fetch courses eligible for feedback submission.
POST	/submit-feedback	Submit course feedback.
GET	/admin/feedback-summary	Retrieve aggregated feedback for admin review.

6 Data Model

6.1 Feedback Table

Field	Type	Description
feedbackId	INT	Unique identifier for feedback.
studentId	INT	ID of the student providing feedback.
courseId	INT	ID of the course.
comments	STRING	Student feedback comments.
rating	INT	Rating provided by student.

createdAt	DATETIME	Timestamp of submission.

7 Security Considerations

- **Access Control:** Ensure only authenticated students can submit feedback.
- **Validation:** Enforce course enrollment verification to prevent misuse.
- **Data Integrity:** Prevent duplicate or tampered submissions.
- **Secure Transmission:** Use HTTPS for all API communications.

8 Inter-Service Integration

1. Student accesses feedback form via frontend portal.
2. Frontend requests eligible courses from backend.
3. Backend queries database to fetch enrolled courses.
4. Student submits feedback.
5. Backend validates course and stores feedback.
6. Backend returns success confirmation.
7. Admin accesses dashboard to review feedback.

9 Error Handling

Error Code	**HTTP**	**Description**
INVALID COURSE	400	Course not found or invalid.
UNAUTHORIZED ACCESS	403	Student not enrolled in the course.
INVALID INPUT	400	Missing or incorrect feedback data.
SERVER ERROR	500	Internal server failure.

10 Configuration Parameters

Parameter	**Default**	**Description**
MAX_FEEDBACK_LENGTH	1000	Maximum allowed comment length.
MIN_RATING	1	Minimum rating value.
MAX_RATING	5	Maximum rating value.

11 Sequence Diagram Observations

- **Feedback Submission Flow:** The student opens the feedback form, and the frontend requests eligible courses. The backend validates enrollment using the database. Upon submission, feedback is inserted into the database, and a success response is returned.
- **Validation Step:** The system ensures that a student can only submit feedback for courses they are enrolled in.
- **Admin Flow:** Admin accesses feedback summary via dashboard, triggering database queries to fetch feedback reports.

12 Class Diagram Responsibilities

- **FeedbackController:** Handles incoming requests for submitting and viewing feedback.
- **FeedbackService:** Orchestrates validation, storage, and retrieval logic.
- **FeedbackRepository:** Manages persistence and retrieval of feedback records.
- **CourseRepository:** Ensures course validation before feedback submission.
- **AdminRepository:** Notifies admin and facilitates review operations.
- **CourseFeedback:** Core entity representing feedback data.
- **Student, Course, Admin:** Supporting entities representing system users and resources.

Academic Section & Admin Management System
Grade Submission & Management Service
Service Documentation — System Design

1 Service Overview

The Grade Submission & Management Service enables faculty members to assign and manage grades for students enrolled in their courses. It ensures secure access control by verifying faculty ownership of courses, supports both course-level and assignment-level grading, and persists grades in the system database. The service facilitates accurate academic evaluation and reporting.

2 Core Responsibilities

- **Grade Submission:** Allow faculty to upload marks and corresponding letter grades.
- **Faculty Authorization:** Ensure only assigned faculty can grade a course.
- **Student Validation:** Verify student existence and enrollment.
- **Assignment Validation:** Validate assignment-related grading when applicable.
- **Grade Calculation:** Convert marks into letter grades.
- **Data Persistence:** Store grades securely in the database.

3 Architecture & Components

3.1 Internal Modules

- **GradeController:** Handles incoming requests for grade submission.
- **GradeService:** Core business logic layer orchestrating validation and grade processing.
- **GradeRepository:** Handles persistence and retrieval of grade data.
- **StudentRepository:** Validates student existence.
- **CourseRepository:** Validates course ownership and existence.
- **AssignmentRepository:** Validates assignment details.

3.2 Entities

- **Grade:** Stores student grades, marks, associated course/assignment, and timestamp.
- **Student:** Represents student details.
- **Course:** Represents course metadata.
- **Assignment:** Represents assignment details (optional).
- **Faculty:** Represents faculty member uploading grades.

4 Grade Lifecycle

State	Trigger	Description
Requested	Course Selected	Faculty selects course for grading.
Validated	Ownership Verified	System verifies faculty is assigned to the course.
Fetched	Student List Retrieved	Enrolled students are fetched from database.
Submitted	Grades Entered	Faculty submits marks and grades.
Stored	Database Insert	Grades are stored for each student.
Completed	Confirmation Displayed	System confirms successful upload.

5 API Endpoints

Method	Endpoint	Description
POST	/submit-grades	Submit grades for a course.
POST	/submit-assignment-grades	Submit assignment-specific grades.
GET	/students-enrolled	Fetch students enrolled in a course.

6 Data Model

6.1 Grade Table

Field	Type	Description
gradeId	INT	Unique identifier for grade.
studentId	INT	ID of the student.
courseId	INT	Associated course ID.
assignmentId	INT	Associated assignment (optional).

marks	FLOAT	Marks obtained by the student.
grade	STRING	Letter grade assigned.
uploadedAt	DATETIME	Timestamp of submission.

7 Security Considerations

- **Authorization:** Ensure only assigned faculty can submit grades.
- **Data Integrity:** Prevent duplicate or inconsistent grade entries.
- **Validation:** Verify student-course relationships before grading.
- **Secure Communication:** Enforce HTTPS for all interactions.

8 Inter-Service Integration

1. Faculty selects a course via grading portal.
2. Frontend requests enrolled students from backend.
3. Backend verifies faculty ownership using database.
4. Backend retrieves enrolled student list.
5. Faculty inputs marks and grades.
6. Backend processes and stores grades.
7. System confirms successful upload and displays report.

9 Error Handling

Error Code	**HTTP**	**Description**
UNAUTHORIZED FACULTY	403	Faculty not assigned to the course.
INVALID STUDENT	400	Student does not exist or not enrolled.
INVALID ASSIGNMENT	400	Assignment not found.
INVALID INPUT	400	Missing or incorrect grade data.
SERVER ERROR	500	Internal server failure.

10 Configuration Parameters

Parameter	Default	Description
MAX_MARKS	100	Maximum marks allowed.
GRADE_BOUNDARIES	-	Mapping of marks to letter grades.
ALLOW_REUPLOAD	FALSE	Whether grade updates are allowed.

11 Sequence Diagram Observations

- **Ownership Validation:** The backend verifies that the faculty member is assigned to the selected course before proceeding.
- **Student Retrieval:** The system queries the database to fetch enrolled students and displays them in the grading interface.
- **Batch Grade Submission:** Grades are submitted as JSON data and inserted into the database iteratively for each student.
- **Confirmation Flow:** Upon successful insertion, a confirmation message is returned and displayed to the faculty.

12 Class Diagram Responsibilities

- **GradeController:** Handles grade submission requests from the frontend.
- **GradeService:** Orchestrates validation, grade calculation, and persistence logic.
- **GradeRepository:** Manages storage and retrieval of grade records.
- **StudentRepository:** Ensures student validation.
- **CourseRepository:** Validates course ownership and existence.
- **AssignmentRepository:** Validates assignment-related grading.
- **Grade:** Core entity representing grading data.
- **Student, Course, Assignment, Faculty:** Supporting entities for system operations.

Academic Section & Admin Management System
Attendance Management Service
Service Documentation — System Design

1 Service Overview

The Attendance Management Service enables faculty members to record and manage attendance for students enrolled in their courses. It ensures proper validation of faculty ownership, verifies student enrollment, and supports efficient bulk recording of attendance data. The service maintains accurate attendance records for academic tracking and reporting.

2 Core Responsibilities

- **Attendance Recording:** Allow faculty to mark attendance for students.
- **Faculty Authorization:** Ensure only assigned faculty can mark attendance.
- **Student Validation:** Verify student enrollment in the course.
- **Bulk Processing:** Support batch insertion and updates for attendance records.
- **Attendance Updates:** Allow modification of previously recorded attendance.

3 Architecture & Components

3.1 Internal Modules

- **AttendanceController:** Handles HTTP requests for marking and updating attendance.
- **AttendanceService:** Core business logic layer managing validation and processing.
- **AttendanceRepository:** Handles persistence and retrieval of attendance records.
- **StudentRepository:** Validates student existence.
- **CourseRepository:** Validates course and faculty ownership.

3.2 Entities

- **Attendance:** Stores attendance records including student ID, course ID, date, and status.
- **Student:** Represents student details.
- **Course:** Represents course metadata.
- **Faculty:** Represents faculty marking attendance.

4 Attendance Lifecycle

State	Trigger	Description
Requested	Course Selected	Faculty selects course for attendance marking.
Validated	Ownership Verified	System verifies faculty assignment to course.
Fetched	Student List Retrieved	Enrolled students are fetched from database.
Marked	Attendance Input	Faculty marks attendance statuses.
Stored	Database Insert	Attendance records are stored in bulk.
Updated	Attendance Modified	Existing records are updated if required.
Completed	Confirmation Displayed	System confirms successful operation.

5 API Endpoints

Method	Endpoint	Description
GET	/course/students	Fetch students enrolled in a course.
POST	/attendance/record	Record attendance for a course.
PUT	/attendance/update	Update attendance status for a student.

6 Data Model

6.1 Attendance Table

Field	Type	Description
attendanceId	INT	Unique identifier for attendance record.
studentId	INT	ID of the student.

courseId	INT	Associated course ID.
date	DATE	Date of attendance.
status	STRING	Attendance status (Present/Absent).

7 Security Considerations

- **Authorization:** Only assigned faculty can mark attendance.
- **Data Integrity:** Prevent duplicate entries for same student-course-date.
- **Validation:** Ensure student is enrolled in the course.
- **Secure Communication:** Use HTTPS for all API interactions.

8 Inter-Service Integration

1. Faculty selects course via portal.
2. Frontend requests enrolled students from backend.
3. Backend verifies faculty ownership using database.
4. Backend retrieves student roster.
5. Faculty marks attendance and submits.
6. Backend performs bulk insert/update operations.
7. System confirms successful attendance recording.

9 Error Handling

Error Code	**HTTP**	**Description**
UNAUTHORIZED FACULTY	403	Faculty not assigned to the course.
INVALID STUDENT	400	Student not found or not enrolled.
DUPLICATE ENTRY	409	Attendance already recorded for the date.
INVALID INPUT	400	Missing or incorrect attendance data.
SERVER ERROR	500	Internal server failure.

10 Configuration Parameters

Parameter	Default	Description
ALLOW_UPDATE	TRUE	Whether attendance updates are permitted.
ATTENDANCE_STATUS	Present/Absent	Allowed status values.
MAX_BATCH_SIZE	200	Maximum number of records per submission.

11 Sequence Diagram Observations

- **Faculty Verification:** Backend validates faculty ownership before allowing attendance operations.
- **Roster Retrieval:** System fetches enrolled students using join queries on enrollment data.
- **Bulk Processing:** Attendance is processed in batch mode using loop-based insert/update operations.
- **Confirmation Flow:** After successful persistence, a confirmation message is returned to the frontend.

12 Class Diagram Responsibilities

- **AttendanceController:** Handles attendance-related API requests.
- **AttendanceService:** Manages validation, enrollment checks, and processing logic.
- **AttendanceRepository:** Handles storage and retrieval of attendance records.
- **StudentRepository:** Validates student details.
- **CourseRepository:** Validates course and faculty assignment.
- **Attendance:** Core entity representing attendance data.
- **Student, Course, Faculty:** Supporting entities.

Academic Section & Admin Management System
Assignment Management Service
Service Documentation — System Design

1 Service Overview

The Assignment Management Service enables faculty members to create, configure, and publish assignments for their courses. It supports defining assignment details such as title, description, due dates, and grading rubrics. The service ensures only authorized faculty can create assignments for their courses and manages structured rubric-based evaluation.

2 Core Responsibilities

- **Assignment Creation:** Allow faculty to create assignments with detailed metadata.
- **Course Validation:** Ensure assignments are created only for valid courses.
- **Rubric Management:** Support creation and association of grading rubrics.
- **Assignment Publishing:** Enable publishing assignments for student access.
- **Data Persistence:** Store assignments and rubrics securely in the database.

3 Architecture & Components

3.1 Internal Modules

- **AssignmentController:** Handles HTTP requests for assignment creation and publishing.
- **AssignmentService:** Core business logic orchestrating validation and persistence.
- **AssignmentRepository:** Manages storage and retrieval of assignments.
- **RubricRepository:** Handles rubric storage and association with assignments.
- **CourseRepository:** Validates course existence and ownership.

3.2 Entities

- **Assignment:** Stores assignment details such as title, description, due date, and status.
- **Rubric:** Defines grading criteria and maximum scores.
- **Course:** Represents course metadata.
- **Faculty:** Represents faculty creating assignments.

4 Assignment Lifecycle

State	Trigger	Description
Requested	Create Assignment Selected	Faculty initiates assignment creation.
Validated	Course Verified	System validates course ownership and existence.
Configured	Details Entered	Assignment details and rubric are provided.
Stored	Database Insert	Assignment and rubric are stored in database.
Published	Publish Action	Assignment is made available to students.

5 API Endpoints

Method	Endpoint	Description
GET	/faculty/courses	Fetch courses assigned to faculty.
POST	/assignment/create	Create a new assignment.
POST	/assignment/publish	Publish an assignment.

6 Data Model

6.1 Assignment Table

Field	Type	Description
assignmentId	INT	Unique identifier for assignment.
courseId	INT	Associated course ID.
facultyId	INT	Creator faculty ID.
title	STRING	Assignment title.
description	STRING	Assignment details.
dueDate	DATETIME	Submission deadline.
maxMarks	INT	Maximum marks.

status	STRING	Assignment status (Draft/Published).

6.2 Rubric Table

Field	Type	Description
rubricId	INT	Unique identifier for rubric.
assignmentId	INT	Associated assignment ID.
criteria	STRING	Evaluation criteria.
maxScore	INT	Maximum score for the criteria.

7 Security Considerations

- **Authorization:** Only assigned faculty can create and publish assignments.
- **Validation:** Ensure assignments are linked to valid courses.
- **Data Integrity:** Prevent invalid or incomplete assignment configurations.
- **Secure Communication:** Use HTTPS for all interactions.

8 Inter-Service Integration

1. Faculty selects assignment creation option via portal.
2. Frontend fetches courses assigned to faculty.
3. Backend retrieves course list from database.
4. Faculty inputs assignment details and rubric.
5. Backend validates course and stores assignment data.
6. Assignment is published and confirmation is returned.

9 Error Handling

Error Code	HTTP	Description
INVALID COURSE	400	Course not found or invalid.
UNAUTHORIZED FACULTY	403	Faculty not assigned to the course.
INVALID INPUT	400	Missing or incorrect assignment data.
SERVER ERROR	500	Internal server failure.

10 Configuration Parameters

Parameter	Default	Description
MAX_TITLE_LENGTH	255	Maximum length of assignment title.
MAX_DESCRIPTION_LENGTH	2000	Maximum description size.
MAX_RUBRIC_ITEMS	10	Maximum rubric criteria allowed.

11 Sequence Diagram Observations

- **Course Fetching:** Backend retrieves courses assigned to faculty before assignment creation.
- **Assignment Creation Flow:** Faculty submits assignment details which are validated and stored.
- **Publishing Flow:** Assignment status is updated and confirmation is returned to frontend.
- **Database Interaction:** Assignment and rubric data are inserted into respective tables.

12 Class Diagram Responsibilities

- **AssignmentController:** Handles assignment-related API requests.
- **AssignmentService:** Manages validation, rubric association, and persistence.
- **AssignmentRepository:** Handles storage and retrieval of assignments.
- **RubricRepository:** Manages rubric data.
- **CourseRepository:** Validates course existence and ownership.
- **Assignment:** Core entity representing assignment data.
- **Rubric:** Represents evaluation criteria.
- **Course, Faculty:** Supporting entities.

Academic Section & Admin Management System
Assignment Submission Review Service
Service Documentation — System Design

1 Service Overview

The Assignment Submission Review Service enables faculty members to review student submissions, download uploaded files, evaluate performance, and provide grades and feedback. It integrates with file storage systems and maintains evaluation records in the database. The service ensures structured and transparent grading workflows.

2 Core Responsibilities

- **Submission Retrieval:** Fetch student submissions for a given assignment.
- **File Access:** Enable viewing and downloading of submitted files.
- **Evaluation:** Allow faculty to assign marks and grades.
- **Feedback Storage:** Store qualitative feedback for each submission.
- **Grade Persistence:** Save evaluation results securely in the database.

3 Architecture & Components

3.1 Internal Modules

- **ReviewController:** Handles API requests for fetching submissions, downloading files, and evaluating submissions.
- **ReviewService:** Core business logic managing validation, retrieval, and evaluation workflows.
- **SubmissionRepository:** Retrieves submission records from database.
- **AssignmentRepository:** Validates assignment details.
- **GradeRepository:** Stores grading results.
- **FeedbackRepository:** Stores faculty feedback.

3.2 Entities

- **Submission:** Represents student submissions including file URL and timestamp.
- **Grade:** Stores marks and grades assigned to submissions.
- **Feedback:** Stores qualitative feedback provided by faculty.
- **Assignment:** Represents assignment metadata.
- **Student:** Represents student details.
- **Faculty:** Represents faculty evaluating submissions.

4 Review Lifecycle

State	Trigger	Description
Requested	Assignment Selected	Faculty selects assignment to review.
Fetched	Submissions Retrieved	System fetches submission list from database.
Viewed	File Accessed	Faculty views/downloads submission files.
Evaluated	Marks Entered	Faculty inputs marks, grades, and feedback.
Stored	Database Insert	Grades and feedback are stored.
Completed	Confirmation Displayed	System confirms evaluation success.

5 API Endpoints

Method	Endpoint	Description
GET	/submissions	Fetch submissions for an assignment.
GET	/submission/file	Download submission file.
POST	/review-submission	Submit evaluation (marks, grade, feedback).

6 Data Model

6.1 Submission Table

Field	Type	Description
submissionId	INT	Unique identifier for submission.
assignmentId	INT	Associated assignment ID.

studentId	INT	ID of submitting student.
fileUrl	STRING	Location of uploaded file.
submittedAt	DATETIME	Submission timestamp.

6.2 Grade Table

Field	**Type**	**Description**
gradeId	INT	Unique identifier for grade.
studentId	INT	Student ID.
assignmentId	INT	Assignment ID.
marks	FLOAT	Marks assigned.
grade	STRING	Letter grade.
uploadedAt	DATETIME	Timestamp of grading.

6.3 Feedback Table

Field	**Type**	**Description**
feedbackId	INT	Unique identifier for feedback.
facultyId	INT	Faculty providing feedback.
studentId	INT	Student receiving feedback.
assignmentId	INT	Associated assignment.
comments	STRING	Feedback text.
createdAt	DATETIME	Timestamp of feedback.

7 Security Considerations

- **Authorization:** Only assigned faculty can review submissions.
- **File Access Security:** Secure file URLs and access permissions.
- **Data Integrity:** Prevent duplicate or tampered evaluations.
- **Secure Communication:** Use HTTPS for all API calls.

8 Inter-Service Integration

1. Faculty selects assignment from dashboard.
2. Frontend requests submissions from backend.
3. Backend retrieves submission list from database.
4. Faculty views/downloads submission files via file server.
5. Faculty submits evaluation data.
6. Backend stores grades and feedback.
7. System confirms evaluation and updates review list.

9 Error Handling

Error Code	HTTP	Description
INVALID ASSIGNMENT	400	Assignment not found.
SUBMISSION NOT FOUND	404	Submission does not exist.
UNAUTHORIZED FACULTY	403	Faculty not assigned to the course.
FILE ACCESS ERROR	500	Unable to retrieve submission file.
INVALID INPUT	400	Missing or incorrect evaluation data.
SERVER ERROR	500	Internal server failure.

10 Configuration Parameters

Parameter	Default	Description
MAX_FILE_SIZE	10MB	Maximum allowed submission size.
ALLOWED_FILE_TYPES	PDF, ZIP, DOCX	Supported submission formats.
ALLOW_REGRADING	TRUE	Whether re-evaluation is allowed.

11 Sequence Diagram Observations

- **Submission Retrieval:** Backend fetches submissions using join queries between submission and student tables.
- **File Access Flow:** Files are retrieved via file server using stored URLs.
- **Evaluation Flow:** Faculty submits marks and feedback which are stored in grade and feedback tables.
- **Confirmation Flow:** System confirms evaluation and refreshes the submission list.

12 Class Diagram Responsibilities

- **ReviewController:** Handles submission-related API requests.
- **ReviewService:** Manages evaluation workflow and validation.
- **SubmissionRepository:** Fetches submission data.
- **AssignmentRepository:** Validates assignment.

- **GradeRepository:** Stores grading results.
- **FeedbackRepository:** Stores feedback data.
- **Submission:** Core entity representing uploaded work.
- **Grade, Feedback:** Evaluation-related entities.
- **Student, Faculty, Assignment:** Supporting entities.

Academic Section & Admin Management System

Performance Analytics Service Documentation

System Design Team

March 31, 2026

1 Service Overview

The Performance Analytics Service provides faculty and administrators with a comprehensive, real-time analytics suite to evaluate student progress. It aggregates data from multiple academic sources to generate visual reports including charts, graphs, and heatmaps, along with class-wide averages and historical trends.

2 Core Responsibilities

- Class-wide average report generation
- Historical performance trend computation
- Detailed attendance statistics and percentage calculations
- Faculty-specific authorization and data filtering

3 Architecture & Components

3.1 Internal Modules

- **PerformanceAnalyticsManager:** Central service responsible for orchestrating data aggregation and analytics computation.
- **Grade Repository:** Fetches student marks and letter grades.
- **Attendance Repository:** Retrieves attendance records (Present/Absent with dates).
- **Assignment Repository:** Provides assignment constraints including maximum marks.

4 Service Lifecycle

State	Trigger	Description
Requested	Admin/Faculty Action	User selects a specific course to view analytics.
Authorized	Ownership Check	Backend verifies faculty ownership of the course.
Fetched	Data Retrieval	Grades, assignments, and attendance records are fetched.
Processed	Logic Computation	Averages, trends, and attendance percentages are computed.
Displayed	UI Render	Charts, graphs, and heatmaps are rendered on the dashboard.

5 API Endpoints

Method	Endpoint	Description
GET	`/api/analytics/course/{course_id}`	Fetch complete analytics (grades + trends + attendance).
GET	`/api/analytics/trends/{course_id}`	Retrieve historical grade trends.
GET	`/api/analytics/attendance/{course_id}`	Fetch attendance statistics and percentages.

6 Data Model

6.1 Course Table

Field	Type	Description
course_id	INT (PK)	Unique course identifier
course_code	STRING	Course code
course_title	STRING	Course title
credit_value	DECIMAL	Credit value of course

6.2 Grade Table

Field	Type	Description
grade_id	INT (PK)	Unique identifier for grade record
student_id	INT (FK)	Reference to student
course_id	INT (FK)	Reference to course
marks	FLOAT	Numerical score
grade	STRING	Letter grade

6.3 Attendance Table

Field	Type	Description
attendance_id	INT (PK)	Unique attendance record
student_id	INT (FK)	Reference to student
course_id	INT (FK)	Reference to course
date	DATE	Date of attendance
status	ENUM	Present / Absent

6.4 Assignment Table

Field	Type	Description
assignment_id	INT (PK)	Unique assignment identifier
course_id	INT (FK)	Reference to course
max_marks	INT	Maximum marks

6.5 Relationships

- One **Course** has many **Grades**
- One **Course** has many **Attendance records**
- One **Course** has many **Assignments**

7 Security Considerations

- **JWT-based Authentication:** Only authenticated users can access APIs.
- **RBAC Authorization:** Faculty can only access their assigned courses.
- **HTTPS Encryption:** Protects sensitive academic data in transit.

8 Inter-Service Integration

1. Frontend sends course ID request.
2. Backend verifies ownership via Course service.
3. Grade, Assignment, and Attendance services return raw data.
4. PerformanceAnalyticsManager aggregates and processes data.
5. JSON response sent for visualization.

9 Error Handling

Error Code	HTTP	Description
UNAUTHORIZED	401	User not authorized for this course
NOT_FOUND	404	No data found for the course
SERVER_ERROR	500	Failure during analytics computation

10 Configuration Parameters

Parameter	Default	Description
ANALYTICS_CACHE	10 minutes	Cache duration for computed analytics
TREND_INTERVAL	Weekly	Granularity of trends
MIN_SAMPLES	1	Minimum data required

11 Sequence Diagram Observations

The system follows a structured sequence:

- Authorization: Verify faculty ownership of course
- Fetch Grades: Retrieve marks and letter grades
- Fetch Assignments: Retrieve maximum marks constraints
- Fetch Attendance: Retrieve attendance logs per student
- Processing: Compute class averages, trends, and attendance percentages

All computations occur at the backend before returning a lightweight JSON response.

12 Class Diagram Responsibilities

- **PerformanceAnalyticsManager:**
  - generateClassAverageReport(course_id)
  - calculateHistoricalTrends(course_id)
  - getAttendanceStatistics(course_id)
- **Entity Relationships:** Course acts as the central entity linking Grade, Attendance, and Assignment tables.
- **Data Integrity:** Foreign keys ensure consistent joins during analytics computation.

Academic Section & Admin Management System

Academic Messaging Service Documentation

System Design Team

March 31, 2026

1 Service Overview

The Academic Messaging Service enables faculty members to communicate directly with students enrolled in their courses. It provides functionality for retrieving enrolled students, sending messages, and allowing students to view received messages through their inbox.

2 Core Responsibilities

- Retrieve enrolled students for a specific course
- Enable faculty to send messages to students
- Store messages with sender, receiver, and timestamp
- Provide inbox functionality for students

3 Architecture & Components

3.1 Internal Modules

- **AcademicMessagingController:** Handles all messaging-related operations including student retrieval, message sending, and inbox fetching.
- **Enrollment Repository:** Resolves enrolled students for a course.
- **Message Repository:** Handles storage and retrieval of messages.
- **Student Repository:** Provides student details (ID, name).

4 Service Lifecycle

State	Trigger	Description
Requested	Faculty Action	Faculty opens messaging interface for a course
Fetched Recipients	API Call	Backend retrieves enrolled students
Composed	Faculty Action	Faculty selects recipient and writes message
Sent	API Call	Message is stored in database
Delivered	Student Action	Student logs in and opens inbox
Displayed	UI Render	Messages displayed in student inbox

5 API Endpoints

Method	Endpoint	Description
GET	`/api/courses/{course_id}/enrolled-students`	Retrieve students enrolled in a course
POST	`/api/messages`	Send a message from faculty to student
GET	`/api/messages/inbox/{student_id}`	Retrieve messages received by a student

6 Data Model

6.1 Faculty Table

Field	Type	Description
faculty_id	INT (PK)	Unique faculty identifier
name	STRING	Faculty name
email	STRING	Faculty email

6.2 Student Table

Field	Type	Description
student_id	INT (PK)	Unique student identifier
name	STRING	Student name
email	STRING	Student email

6.3 Message Table

Field	Type	Description
message_id	INT (PK)	Unique message identifier
sender_id	INT (FK → Faculty)	ID of sender (faculty)
receiver_id	INT (FK → Student)	ID of receiver (student)
message_text	TEXT	Message content
sent_at	DATETIME	Timestamp of message

6.4 Enrollment Table

Field	Type	Description
enrollment_id	INT (PK)	Unique enrollment record
student_id	INT (FK)	Reference to student
course_id	INT (FK)	Reference to course

6.5 Relationships

- One **Faculty** sends many **Messages**
- One **Student** receives many **Messages**
- One **Student** has many **Enrollments**
- Enrollment links **Student** and **Course**
- Faculty is associated with courses (ownership context for messaging authorization)

7 Security Considerations

- **Authentication:** Only authenticated faculty and students can access the system
- **Authorization:** Faculty can only message students enrolled in their courses
- **Data Privacy:** Messages are securely stored and transmitted via HTTPS

8 Inter-Service Integration

1. Frontend requests enrolled students for a course

2. Backend queries Enrollment JOIN Student tables
3. Faculty sends message via API
4. Backend inserts message into Message table
5. Student retrieves messages via inbox API

9 Error Handling

Error Code	HTTP	Description
UNAUTHORIZED	401	User not authenticated
FORBIDDEN	403	Faculty not allowed to message this student
NOT_FOUND	404	Student or course not found
SERVER_ERROR	500	Failed to process message

10 Sequence Diagram Observations

The system follows a three-stage flow:

- **Load Recipients:**
  - Frontend requests enrolled students
  - Backend queries Enrollment JOIN Student tables
  - Returns student directory
- **Compose and Send Message:**
  - Faculty selects student and sends message
  - POST request contains sender_id, receiver_id, message_text
  - Backend inserts into Message table
  - Returns **201 Created** with generated `message_id`
- **Student Receives Message:**
  - Student logs into portal and opens inbox
  - Backend queries Message table WHERE receiver_id = student_id
  - Messages returned sorted by sent_at

All operations are handled by the Backend API before returning structured JSON responses to the frontend.

11 Class Diagram Responsibilities

- **AcademicMessagingController:**
  - getEnrolledStudents(course_id)
  - sendMessage(sender_id, receiver_id, text)
  - getInbox(user_id)
- **Message Entity:** Stores sender, receiver, message content, and timestamp.
- **Entity Relationships:**
  - Message connects Faculty (sender) and Student (receiver)
  - Enrollment resolves eligible recipients
- **Data Integrity:** Foreign key relationships ensure valid sender, receiver, and enrollment mappings.

Academic Section & Admin Management System

Resource Sharing Service Documentation

System Design Team

March 31, 2026

1 Service Overview

The Resource Sharing Service enables faculty members to upload and share learning materials (such as PDFs and slides) with students enrolled in their courses. Uploaded resources are stored in a file storage system and linked to courses and faculty. Students are notified when new resources are available.

2 Core Responsibilities

- Allow faculty to upload course-specific resources
- Store files securely and generate accessible file URLs
- Maintain resource metadata in the database
- Associate resources with courses and faculty
- Notify enrolled students about new resources
- Allow students to view shared resources

3 Architecture & Components

3.1 Internal Modules

- **ResourceController:** Handles resource upload and retrieval operations.
- **File Storage Service:** Stores uploaded files (PDFs, slides) and returns file URLs.
- **Resource Repository:** Stores metadata of uploaded resources.
- **Enrollment Repository:** Retrieves students enrolled in a course.
- **Notification Service:** Sends notifications to students about new resources.

4 Service Lifecycle

State	Trigger	Description
Requested	Faculty Action	Faculty selects course and uploads resource
Uploaded	API Call	File is sent to backend for processing
Stored	File Handling	File is stored and file URL is generated
Recorded	Database Insert	Resource metadata saved in database
Notified	System Trigger	Enrolled students are iteratively notified
Displayed	UI Render	Resource shown as "Shared with Class"

5 API Endpoints

Method	Endpoint	Description
POST	`/resources`	Upload a resource (file + course_id + faculty_id)

6 Data Model

6.1 Faculty Table

Field	Type	Description
faculty_id	INT (PK)	Unique faculty identifier
name	STRING	Faculty name
email	STRING	Faculty email
department	STRING	Department name
designation	STRING	Faculty designation

6.2 Course Table

Field	Type	Description
course_id	INT (PK)	Unique course identifier
course_name	STRING	Name of course
course_code	STRING	Course code
semester	STRING	Semester information

6.3 Student Table

Field	Type	Description
student_id	INT (PK)	Unique student identifier
name	STRING	Student name
email	STRING	Student email
year	INT	Academic year

6.4 Resource Table

Field	Type	Description
resource_id	INT (PK)	Unique resource identifier
faculty_id	INT (FK → Faculty)	Uploader faculty
course_id	INT (FK → Course)	Associated course
file_url	STRING	URL of stored file
uploaded_at	DATETIME	Upload timestamp

6.5 Enrollment Table

Field	Type	Description
enrollment_id	INT (PK)	Unique enrollment record
student_id	INT (FK → Student)	Reference to student
course_id	INT (FK → Course)	Reference to course
enrollment_date	DATE	Date of enrollment

6.6 Relationships

- One **Faculty** teaches one or more **Courses**
- One **Faculty** uploads many **Resources**

- One **Course** has many **Resources**
- Resources are categorized under a **Course**
- One **Student** participates in many **Enrollments**
- One **Course** has many **Enrollments**
- Enrollment links **Student** and **Course**

7 Security Considerations

- **Authentication:** Only authorized faculty can upload resources
- **Authorization:** Faculty can upload only for courses they teach
- **Secure Storage:** Files stored securely with controlled access
- **Data Privacy:** File URLs protected from unauthorized access

8 Inter-Service Integration

1. Frontend sends resource upload request
2. Backend sends file to File Storage Service
3. File Storage returns file URL
4. Backend inserts resource metadata into database
5. Backend retrieves enrolled students using Enrollment table
6. Notification service iterates over students and sends alerts

9 Error Handling

Error Code	HTTP	Description
UNAUTHORIZED	401	User not authenticated
FORBIDDEN	403	Faculty not allowed to upload for this course
BAD_REQUEST	400	Invalid file or missing parameters
SERVER_ERROR	500	File upload or storage failure

10 Sequence Diagram Observations

The system follows the below flow:

- **Upload Resource:**
  - Faculty selects course and uploads file
  - POST request contains file, course_id, faculty_id
- **File Handling:**
  - Backend sends file to File Storage Service
  - File is stored (PDF/Slides)
  - File URL is returned
- **Database Storage:**
  - INSERT into Resource table (course_id, faculty_id, file_url, timestamp)
- **Student Resolution:**
  - SELECT student_id FROM Enrollment WHERE course_id = ?
- **Notification:**
  - System iterates over enrolled students

  - Notify: "New material available for course"

All processing is handled by the backend before confirming successful upload to the frontend.

11 Class Diagram Responsibilities

- **Faculty:** Uploads resources using uploadResource(course_id, file)
- **Student:** Views resources using viewResource(resource_id)
- **Resource:** Stores file metadata and links faculty and course
- **Enrollment:** Connects students to courses
- **Course:** Acts as central entity linking resources and enrollments

Academic Section & Admin Management System

Enrollment Oversight Service Documentation

System Design Team

March 31, 2026

1 Service Overview

The Enrollment Oversight Service allows faculty and administrators to manage course enrollments. It provides functionality to view enrolled students, search for students, enroll new students into a course, and remove students from a course.

2 Core Responsibilities

- View course roster (list of enrolled students)
- Search students by name or identifier
- Enroll students into courses
- Remove (drop) students from courses
- Maintain enrollment records with timestamps

3 Architecture & Components

3.1 Internal Modules

- **EnrollmentOversightManager:** Handles enrollment operations including viewing, adding, and removing students.
- **Enrollment Repository:** Stores and retrieves enrollment records.
- **Student Repository:** Handles student search operations.
- **Course Repository:** Provides course-related data.

4 Service Lifecycle

State	Trigger	Description
Requested	Faculty/Admin Action	User selects course to view roster
Fetched	API Call	Backend retrieves enrolled students
Displayed	UI Render	Course roster displayed
Searched	API Call	Student search results retrieved
Enrolled	API Call	New enrollment record created
Updated	UI Update	UI reflects added student
Removed	API Call	Enrollment record deleted
Updated	UI Update	UI reflects removed student

5 API Endpoints

Method	Endpoint	Description
GET	`/api/courses/{course_id}/enrollments`	Retrieve enrolled students for a course
GET	`/api/students/search?q={query}`	Search students by name or roll number
POST	`/api/enrollments`	Enroll a student into a course
DELETE	`/api/enrollments/{enrollment_id}`	Remove a student from a course

6 Data Model

6.1 Course Table

Field	Type	Description
course_id	INT (PK)	Unique course identifier
course_code	STRING	Course code
course_title	STRING	Course title

6.2 Student Table

Field	Type	Description
student_id	INT (PK)	Unique student identifier
name	STRING	Student name
email	STRING	Student email
department	STRING	Student department

6.3 Enrollment Table

Field	Type	Description
enrollment_id	INT (PK)	Unique enrollment identifier
student_id	INT (FK → Student)	Reference to student
course_id	INT (FK → Course)	Reference to course
enrollment_date	DATE	Date of enrollment

6.4 Relationships

- One **Course** contains many **Enrollments**
- One **Student** participates in many **Enrollments**
- Enrollment links **Student** and **Course**

7 Security Considerations

- **Authentication:** Only authorized faculty/admin users can manage enrollments
- **Authorization:** Users can only manage enrollments for permitted courses
- **Data Integrity:** Prevent duplicate enrollments for the same student and course

8 Inter-Service Integration

1. Frontend requests course roster
2. Backend queries Enrollment JOIN Student tables
3. Student search queries Student table

4. Enrollment creation inserts into Enrollment table
5. Enrollment deletion removes record from database

9 Error Handling

Error Code	HTTP	Description
UNAUTHORIZED	401	User not authenticated
FORBIDDEN	403	User not allowed to manage this course
NOT_FOUND	404	Student or enrollment record not found
CONFLICT	409	Student already enrolled in course
SERVER_ERROR	500	Failed to process enrollment operation

10 Sequence Diagram Observations

The system follows three main flows:

- **View Course Roster:**
  - GET request to retrieve enrollments for a course
  - Backend performs JOIN between Enrollment and Student tables
  - Returns JSON list of students (student_id, name, email, enrollment_date)
- **Enroll a Student:**
  - Faculty/Admin searches for student using query
  - Backend queries Student table (name LIKE ?)
  - POST request with payload {student_id, course_id}
  - INSERT into Enrollment table (student_id, course_id, enrollment_date)
  - Returns 201 Created with enrollment_id
- **Drop a Student:**
  - DELETE request with enrollment_id
  - DELETE FROM Enrollment WHERE enrollment_id = ?
  - Returns 200 OK confirmation

All processing occurs in the backend before updating the frontend UI.

11 Class Diagram Responsibilities

- **EnrollmentOversightManager:**
  - viewCourseRoster(course_id)
  - enrollStudent(student_id, course_id)
  - dropStudent(enrollment_id)
- **Enrollment:** Acts as the central entity linking students and courses.
- **Entity Relationships:**
  - Course contains multiple enrollments
  - Student participates in enrollments
- **Data Integrity:** Foreign keys ensure valid student and course references.

Academic Section & Admin Management System

Student Database Management Service Documentation

System Design Team

March 31, 2026

1 Service Overview

The Student Database Management Service allows administrators to manage student records, including creating new student accounts, updating student information, and deactivating student records. The system separates authentication data, personal information (PII), and academic data across multiple tables to ensure modularity and data integrity.

2 Core Responsibilities

- Create new student records across multiple data layers
- Maintain user credentials and account status
- Manage personal profile information (PII)
- Manage academic student data (department, year)
- Update existing student records
- Deactivate student accounts using soft delete

3 Architecture & Components

3.1 Internal Modules

- **StudentDatabaseManager:** Handles creation, update, and deactivation of student records.
- **User Repository:** Manages authentication credentials and account status.
- **PersonProfile Repository:** Stores personal identifiable information.
- **Student Repository:** Stores academic details of students.

4 Service Lifecycle

State	Trigger	Description
Requested	Admin Action	Admin fills new student form
Created	API Call	New user, profile, and student records created
Displayed	UI Render	Student list updated with new record
Updated	API Call	Existing student data modified
Displayed	UI Render	Updated student information displayed
Deactivated	API Call	Student account marked as inactive
Displayed	UI Render	Student removed or grayed out in UI

5 API Endpoints

Method	Endpoint	Description
POST	`/api/admin/students`	Create a new student record
PUT	`/api/admin/students/{student_id}`	Update an existing student record
DELETE	`/api/admin/students/{student_id}`	Deactivate a student account (soft delete)

6 Data Model

6.1 User Table

Field	Type	Description
user_id	INT (PK)	Unique user identifier
email	STRING	User email
password_hash	STRING	Hashed password
account_status	STRING	Account status (active/disabled)

6.2 PersonProfile Table

Field	Type	Description
person_id	INT (PK)	Unique profile identifier
user_id	INT (FK → User)	Associated user
first_name	STRING	First name
last_name	STRING	Last name
date_of_birth	DATE	Date of birth
address	TEXT	Address
phone	STRING	Contact number

6.3 Student Table

Field	Type	Description
student_id	INT (PK)	Unique student identifier
user_id	INT (FK → User)	Associated user account
name	STRING	Student name
department	STRING	Academic department
year	INT	Academic year

6.4 Relationships

- One **User** has one **PersonProfile**
- One **User** maps to one **Student**
- User manages authentication credentials
- PersonProfile manages personal identifiable information (PII)
- Student stores academic data

7 Security Considerations

- **Authentication:** Only authorized admins can manage student records
- **Password Security:** Passwords stored as hashes

- **Soft Delete:** Accounts are disabled instead of deleted to preserve history
- **PII Protection:** Personal data stored separately for security and compliance

8 Inter-Service Integration

1. Admin submits student creation form
2. Backend inserts into User table and retrieves user_id
3. Backend inserts into PersonProfile table
4. Backend inserts into Student table
5. Updates propagate to UI
6. Updates modify PersonProfile and Student tables
7. Deactivation updates User account_status

9 Error Handling

Error Code	HTTP	Description
UNAUTHORIZED	401	Admin not authenticated
FORBIDDEN	403	Not permitted to manage student data
BAD_REQUEST	400	Invalid input data
CONFLICT	409	Duplicate email or user exists
SERVER_ERROR	500	Failed to process database operations

10 Sequence Diagram Observations

The system follows three main flows:

- **Add New Student:**
  - POST request with payload {email, password, first_name, last_name, dob, department, year}
  - INSERT into User table (email, password_hash, account_status)
  - Retrieve user_id
  - INSERT into PersonProfile (user_id, first_name, last_name, dob, address, phone)
  - INSERT into Student (user_id, name, department, year)
  - Returns 201 Created with student_id
- **Edit Student Record:**
  - PUT request with updated fields
  - UPDATE PersonProfile SET address=?, phone=? WHERE user_id=?
  - UPDATE Student SET department=?, year=? WHERE student_id=?
  - Returns 200 OK confirmation
- **Deactivate Student:**
  - DELETE request with student_id
  - Fetch associated user_id from Student table
  - UPDATE User SET account_status = 'disabled' WHERE user_id=?
  - Returns 200 OK confirmation

  All operations are handled in the backend before updating the UI.

11 Class Diagram Responsibilities

- **StudentDatabaseManager:**
  - addStudent(userData, profileData, studentData)
  - editStudentRecord(student_id, updates)
  - deactivateStudent(student_id)
- **User:** Stores authentication credentials and account status
- **PersonProfile:** Stores personal identifiable information
- **Student:** Stores academic information
- **Data Integrity:** Ensures consistency across User, PersonProfile, and Student tables using foreign keys

Academic Section & Admin Management System
Faculty Account Administration Service

Service Documentation System Design

1 Service Overview

The Faculty Account Administration Service is responsible for managing the lifecycle of faculty credentials and professional profiles within the institution. It enables administrators to provision new accounts, update professional designations, and manage account accessibility to ensure secure and up-to-date faculty records.

2 Core Responsibilities

- **Account Provisioning:** Create integrated user login credentials and detailed faculty profiles.
- **Profile Management:** Update faculty designations and link faculty members to academic departments.
- **Lifecycle Control:** Deactivate faculty accounts for departing members to maintain system security.
- **Data Validation:** Ensure department assignments are validated against existing institutional structures.

3 Architecture & Components

3.1 Internal Modules

- **FacultyAdminController:** Primary entry point for administrative commands and request delegation.
- **User Entity:** Manages core authentication data including email and account status.
- **Faculty Entity:** Stores professional attributes such as department and designation.
- **Department Entity:** Maintains a registry of valid institutional departments for validation.

4 Administration Lifecycle

State	Trigger	Description
Provisioned	Add New Faculty	User account and profile are inserted into the DB.
Active	Profile Confirmed	Account status is set to 'active' for system access.
Updated	Edit Details	Faculty designation or department links are modified.
Disabled	Deactivate Account	Account status set to 'disabled' to block access.

5 API Endpoints

Method	Endpoint	Description
POST	/api/admin/faculty	Create a new faculty user and profile.
PUT	/api/admin/faculty/{id}	Update designation and department links.
PATCH	/api/admin/faculty/{id}/status	Modify account status (e.g., to 'disabled').

6 Data Model

6.1 Faculty Database Entry (MySQL)

Field	Type	Description
faculty_id	INT [PK]	Internal unique identifier for faculty.
name	VARCHAR	Full name of the faculty member.
department	VARCHAR	Linked academic department.
designation	VARCHAR	Professional rank (e.g., Associate Prof).
account_status	VARCHAR	Current state (active/disabled).

7 Security Considerations

- **Password Safety:** Passwords are never stored as plain text; the system utilizes `password_hash`.
- **Relational Integrity:** Faculty updates require validation against the `departments` table.
- **Access Revocation:** Account disabling immediately triggers an update to the core `Users` table.

8 Inter-Service Integration

1. Admin submits faculty details via the Admin Portal.
2. Backend API processes the payload and initiates a transaction in the Database.
3. The system first creates a `User` record, followed by a `Person Profile`, and finally a `Faculty` record.
4. Success messages and UI directory updates are returned to the Admin Portal.

9 Error Handling

Error Case	HTTP	Description
INVALID_DEPT	400	The specified department does not exist.
DUPLICATE_USER	409	Email already exists in the Users table.
DB_TIMEOUT	503	Database fails to confirm profile creation.

10 Configuration Parameters

Parameter	Default	Description
DEFAULT_STATUS	active	Initial state of a newly created account.
HASH_ALGORITHM	BCrypt	Cryptographic method for password hashing.
DEPT_VALIDATION	enabled	Boolean flag to enforce department checks.

11 Sequence Diagram Observations

- **Multi-Step Provisioning:** Account creation requires three distinct SQL inserts (Users, Profile, Faculty) to maintain normalization.
- **Update Logic:** Designation updates include a validation step to ensure the department exists before updating the record.
- **Deactivation Flow:** Disabling an account involves querying by the faculty ID to find the associated email before updating the `Users` table.

12 Class Diagram Responsibilities

- **FacultyAdminController:** Orchestrates account creation, designation updates, and account disabling.
- **User:** Manages the core login credentials and `account_status`.
- **Faculty:** Manages professional data and its assignment to a specific department.
- **Department:** Acts as the source of truth for valid institutional units.

Academic Section & Admin Management System
Automated Transcript Generation Service

Service Documentation System Design

1 Service Overview

The Automated Transcript Generation Service is responsible for the systematic aggregation of a student's academic history to produce official university transcripts. It automates the retrieval of course records and grades, performs complex CGPA calculations, and generates a formatted, secure PDF document for students or administrators.

2 Core Responsibilities

- **Academic History Retrieval:** Query student and personal profile tables to fetch foundational academic details.
- **Grade Aggregation:** Fetch course codes, titles, credit values, and letter grades for all enrolled courses.
- **GPA Calculation:** Calculate semester-wise GPA and cumulative CGPA based on credit values and final grades.
- **Document Formatting:** Apply official university templates, including logos and watermarks, to the generated transcript.
- **File Delivery:** Deliver the final transcript as a secure PDF byte stream or download link to the user.

3 Architecture & Components

3.1 Internal Modules

- **TranscriptGenerator:** The core service orchestrator that fetches history, calculates GPA, and initiates PDF generation.
- **Backend API:** Acts as the interface between the portal and database, coordinating the data processing flow.
- **PDF Generation Service:** A specialized module that receives JSON data and outputs a formatted PDF file.
- **Database Entities:** Includes `Student`, `Course`, and `Grade` models for structured academic data.

4 Generation Lifecycle

State	Trigger	Description
Requested	Generate Click	User initiates request via the Portal Interface.
Data-Fetched	DB Query	Student profile and all course/grade records retrieved.
Processed	Computation	Results grouped by semester; CGPA/GPA calculated.
Formatted	PDF Template	Data converted to PDF with official university logos.
Delivered	Byte Stream	File payload returned to the portal for download.

5 API Endpoints

Method	Endpoint	Description
GET	/api/transcripts/generate/{id}	Initiate transcript generation for a specific student.
GET	/api/transcripts/history/{id}	Fetch raw academic history for internal processing.

6 Data Model

6.1 Academic Records (MySQL)

Field	Type	Description
student_id	INT [PK]	Unique identifier for the student.
course_code	VARCHAR	Institutional code for the course.
credit_value	DECIMAL	Credit weighting for the course.
marks	FLOAT	Numerical marks obtained by the student.
grade	STRING	Final letter grade issued for the course.

7 Security Considerations

- **Official Templates:** All generated PDFs include watermarks to prevent forgery.
- **Data Privacy:** The Backend API ensures that transcript generation is restricted to authorized student IDs only.
- **Stateless Delivery:** The PDF is delivered as a byte stream to avoid local storage of sensitive records on the generation server.

8 Inter-Service Integration

1. User clicks "Generate Official Transcript" on the Portal Interface.
2. Backend API queries the Database for Student, Person Profile, and Grade tables.
3. API processes data and sends a JSON structure to the PDF Generation Service.
4. PDF Service returns the generated file stream to the Backend API.
5. Final document is pushed to the client for download.

9 Error Handling

Error Code	HTTP	Description
STUDENT_NOT_FOUND	404	Provided Student ID does not exist in the system.
INCOMPLETE_RECORDS	422	Missing grades or credits prevent CGPA calculation.
PDF_GEN_FAILURE	500	The generation service could not apply the template.

10 Configuration Parameters

Parameter	Default	Description
LOGO_PATH	/assets/logo.png	Path to the official university logo.
GPA_PRECISION	2 decimal	The rounding factor for CGPA calculation.
FILE_NAME_PREFIX	Transcript_	Prefix used for the generated PDF file.

11 Sequence Diagram Observations

- **Data Aggregation:** The process involves a complex query across `Grade`, `Course`, and `Enrollment` tables based on student_id.
- **Processing Logic:** GPA/CGPA calculation is performed within the Backend API after grouping records by semester.
- **Template Application:** Logos and watermarks are applied in the final stage by the PDF Generation Service before the file is delivered.

12 Class Diagram Responsibilities

- **TranscriptGenerator:** Responsible for calling `fetchAcademicHistory()`, `calculateCGPA()`, and `generatePDFTranscript()`.
- **Student:** Stores name, department, and current year for the transcript header.
- **Course:** Provides the `credit_value` essential for GPA calculation.
- **Grade:** Connects a student to a course and stores the numerical `marks` and `grade`.

Academic Section & Admin Management System
Certificate Approval Workflow Service

Service Documentation System Design

1 Service Overview

The Certificate Approval Workflow Service manages the digital clearance process for students, specifically focusing on the "No Dues" certification. It provides an automated pipeline for Hostel Staff (Caretakers and Wardens) to review student academic and financial status, provide digital sign-offs, and trigger official notifications upon completion of the clearance cycle.

2 Core Responsibilities

- **Queue Management:** Maintain and display a live queue of No Dues forms awaiting administrative review.
- **Identity Verification:** Verify the identity and account status of students requesting clearance.
- **Multi-Level Approval:** Support sequential sign-offs from both Caretakers and Wardens.
- **Digital Signature Collection:** Securely record signature hashes to validate the approval action.
- **Status Tracking:** Evaluate the overall document status (e.g., 'Pending' to 'Completed') as signatures are collected.

3 Architecture & Components

3.1 Internal Modules

- **CertificateApprovalWorkflow:** Core controller orchestrating request retrieval and sign-off logic.
- **Approval Dashboard:** Frontend interface for Admin/Warden to navigate pending approvals.
- **Backend API:** Coordinates database queries and evaluates document completion states.
- **Notification Service:** Triggers email/SMS alerts to students once approval is finalized.

4 Approval Lifecycle

State	Trigger	Description
Pending	Form Submission	Form appears in queue where Warden/Caretaker Sign is NULL.
In-Review	Select Form	Admin views detailed student profile and form status.
Signed	Admin Sign-off	Database updates specific signature field with hash.
Completed	All Signs Present	System evaluates that all required signatures are collected.
Notified	Completion	Notification queued and sent to the student.

5 API Endpoints

Method	Endpoint	Description
GET	/api/admin/approvals/pending	Fetch JSON array of forms awaiting signatures.
GET	/api/admin/approvals/{form_id}	Retrieve detailed student profile and previous sig
PATCH	/api/admin/approvals/{form_id}/sign	Submit approval action and signature hash.

6 Data Model

6.1 No Dues Database Entry (MySQL)

Field	Type	Description
ID	INT [PK]	Unique identifier for the clearance form.
Roll_No	STRING [FK]	Link to the student requesting clearance.
Caretaker_Sign	STRING	Recorded signature hash for the Caretaker level.
Warden_Sign	STRING	Recorded signature hash for the Warden level.
Account_Status	STRING	Current standing of the student account.

7 Security Considerations

- **Approver Validation:** The system validates the Staff_ID and Role before accepting a sign-off action.
- **Signature Integrity:** Sign-offs are stored as `signature_hash` to prevent tampering.
- **Role-Based Logic:** The API distinguishes between 'Warden' and 'Caretaker' actions in the sign-off payload.

8 Inter-Service Integration

1. Admin navigates to the "Pending Approvals" tab in the Approval Dashboard.
2. Backend API queries the `No_Dues_Form` table for records with null signature fields.
3. Upon sign-off, the Backend updates the database and evaluates the total signature count.
4. Backend triggers the Notification Service to alert the student via Email/SMS.
5. The document is removed from the pending queue and a success message is shown to the Admin.

9 Error Handling

Error Code	HTTP	Description
FORM_NOT_FOUND	404	The specified No Dues form ID does not exist.
UNAUTHORIZED_SIGNER	403	User role does not match the required approver level.
MISSING_SIGNATURE	400	Sign-off attempt failed due to missing signature hash.

10 Configuration Parameters

Parameter	Default	Description
REQUIRED_SIGNS	2	Total signatures (Caretaker + Warden) for completion.
NOTIF_CHANNEL	Email/SMS	Primary channels for student approval alerts.
QUEUE_REFRESH_MS	30000	Automatic refresh interval for the pending queue.

11 Sequence Diagram Observations

- **Query Logic:** Pending requests are filtered where `Warden_Sign IS NULL` OR `Caretaker_Sign IS NULL`.
- **Aggregated Details:** Reviewing a request fetches both the form status and the detailed student profile (Name, Status) in a single view.
- **Completion Check:** After every signature update, the system performs a self-check to evaluate if the document status should move to 'Completed'.

12 Class Diagram Responsibilities

- **CertificateApprovalWorkflow:** Orchestrator responsible for `getPendingRequests()` and `signOffDocument()`.
- **Hostel_Staff:** Stores `Staff_ID` and `Role` for validating authorized approvers.
- **No_Dues_Form:** Central entity tracking student reference and collected sign-offs.
- **Student:** Provides core identity data (`Roll_No`, `Name`) for clearance verification.

Academic Section & Admin Management System
Program Configuration Service

Service Documentation System Design

1 Service Overview

The Program Configuration Service is responsible for defining and maintaining the academic structure of the institution. It allows administrators to configure new academic programs, link them to specific departments, and manage core attributes such as degree levels and semester durations.

2 Core Responsibilities

- **Program Definition:** Create new academic programs with unique codes, names, and degree levels.
- **Departmental Mapping:** Link programs to active departments to maintain institutional hierarchy.
- **Configuration Management:** Update existing program durations and degree levels as curriculum requirements evolve.
- **Audit & Visibility:** Provide a comprehensive view of all configured programs for administrative auditing.

3 Architecture & Components

3.1 Internal Modules

- **ProgramConfigurationManager:** Core controller handling the logic for defining and updating academic programs.
- **Admin Dashboard:** Frontend interface used by administrators to input program details and view lists.
- **Backend API:** Coordinates requests between the dashboard and the database, including prerequisite data loading.
- **Program Entity:** Stores attributes like `program_code`, `degree_level`, and `duration_semesters`

4 Configuration Lifecycle

State	Trigger	Description
Initialized	Load Prerequisites	System fetches active departments for the selection dropdown.
Validated	Define Program	Backend verifies that the selected `department_id` exists.
Configured	INSERT Successful	New program record is saved and assigned a `program_id`.
Audited	View List	Admin renders a data table of all current academic programs.
Updated	Edit Configuration	Changes to duration or degree level are persisted to the DB.

5 API Endpoints

Method	Endpoint	Description
GET	/api/departments?active_flag=true	Retrieve list of active departments for selection.
POST	/api/programs	Create a new academic program entry.
GET	/api/programs?include=departments	Fetch detailed program list with department joins.
PUT	/api/programs/{program_id}	Update duration or degree level for a program.

6 Data Model

6.1 Program Database Entry (MySQL)

Field	Type	Description
program_id	INT [PK]	Internal unique identifier for the program.
program_code	STRING	Institutional code (e.g., CSE, ME).
degree_level	STRING	Level of study (e.g., B.Tech, M.Tech).
duration_semesters	INT	Total number of semesters in the program.
department_id	INT [FK]	Link to the offering Department.

7 Security Considerations

- **Input Validation:** The Backend API strictly validates `department_id` existence before insertion.
- **Role-Based Access:** Configuration actions are restricted to the Admin Dashboard level.
- **Relational Integrity:** Foreign key constraints (`department_id`) ensure programs are always tied to a valid department.

8 Inter-Service Integration

1. Admin navigates to the "Program Configuration" section of the dashboard.
2. Dashboard fetches active departments from the Backend API.
3. Admin submits new program details; Backend performs a DB validation and INSERT.
4. Backend returns a 201 Created status and updates the UI program list.
5. Modifications to existing programs trigger a PUT request to update specific fields.

9 Error Handling

Error Code	HTTP	Description
INVALID_DEPT	400	The provided `department_id` does not exist.
DUPLICATE_CODE	409	A program with the same code already exists.
MISSING_FIELDS	400	Required attributes like `degree_level` were not provided.

10 Configuration Parameters

Parameter	Default	Description
ACTIVE_FLAG	true	Filter used to load current departments.
MIN_DURATION	1	Minimum allowable semesters for a program.
MAX_DURATION	12	Maximum allowable semesters for a program.

11 Sequence Diagram Observations

- **Prerequisite Loading:** The workflow begins by loading active departments to populate the dropdown menu.
- **Relational Queries:** Viewing programs uses a JOIN operation with the Departments table for detailed rendering.
- **Atomic Updates:** Edit operations (`PUT`) target specific attributes like `duration_semesters` or `degree_level`.

12 Class Diagram Responsibilities

- **ProgramConfigurationManager:** Orchestrates the `defineProgram()` and `listAllPrograms()` operations.
- **Department:** Stores departmental identifiers and active status flags.
- **Program:** Encapsulates individual program data and links to a specific department via FK.

Academic Section & Admin Management System
Course Catalog Control Service

Service Documentation System Design

1 Service Overview

The Course Catalog Control Service manages the master registry of all academic courses offered by the institution. It provides administrative tools to provision new courses, update credit weightings, and manage the availability of curriculum items across different departments. This service ensures that the academic offerings remain structured and validated against institutional standards.

2 Core Responsibilities

- **Curriculum Provisioning:** Add new courses with specific codes, titles, and credit values.
- **Credit Management:** Update existing course credit values to reflect curriculum changes.
- **Lifecycle Control:** Toggle the active status of courses to manage their visibility in the catalog.
- **Departmental Mapping:** Ensure every course is correctly linked and validated against an existing department.
- **Uniqueness Enforcement:** Prevent duplicate course codes within the institutional database.

3 Architecture & Components

3.1 Internal Modules

- **CourseCatalogController:** The primary entry point handling requests for course creation and modification.
- **Course (Entity):** Core data model maintaining course attributes like `course_code`, `credit_value`, and `active_flag`.
- **Department (Entity):** Validation module ensuring courses are offered by legitimate institutional units.
- **Catalog Controller (Logic):** Orchestrates the validation phase to check for existing codes before DB insertion.

4 Course Lifecycle

State	Trigger	Description
Requested	"Add New Course"	Admin submits course details via the UI.
Validated	Code Check	System confirms the course code is unique.
Active	INSERT Successful	Course is added to catalog with `active_flag` set to true.
Modified	Credit Update	Existing credit values are adjusted via controller.
Inactive	Status Toggle	Course remains in DB but is hidden from active catalog.

5 API Endpoints

Method	Endpoint	Description
POST	/api/catalog/courses	Create a new course entry in the system.
PUT	/api/catalog/credits	Update credit value for a specific course ID.
PATCH	/api/catalog/status	Toggle the active/inactive status of a course.

6 Data Model

6.1 Course Database Entry (MySQL)

Field	Type	Description
course_id	INT [PK]	Internal unique identifier for the course.
course_code	VARCHAR	Unique institutional code (e.g., CS101).
credit_value	DECIMAL	Academic weighting of the course.
department_id	INT [FK]	Link to the offering department.
active_flag	BOOLEAN	Flag for catalog visibility.

7 Security Considerations

- **Administrative Authorization:** Only users with Academic Admin roles can access the Catalog Controller.
- **Relational Integrity:** Foreign key constraints ensure a course cannot exist without a valid `department_id`.
- **Input Validation:** The Validation Phase prevents duplicate `course_code` entries to maintain data consistency.

8 Inter-Service Integration

1. Admin selects "Add New Course" and submits details through the Admin Portal UI.
2. Portal UI issues a POST request to the Catalog Controller.
3. Controller queries the Database to check if the course code already exists.
4. If unique, the Controller inserts the record and returns a success confirmation.
5. The Admin Portal displays a "Course Created Successfully" notification.

9 Error Handling

Error Code	HTTP	Description
DUPLICATE_CODE	409	The submitted course code already exists in the system.
INVALID_DEPT	400	The `department_id` provided does not exist.
MALFORMED_DATA	400	Missing required fields such as title or credit value.

10 Configuration Parameters

Parameter	Default	Description
DEFAULT_ACTIVE	true	Default state for new course entries.
CREDIT_PRECISION	1	Number of decimal places for credit values.
CODE_PATTERN	[A-Z0-9]+	Regex for validating course code format.

11 Sequence Diagram Observations

- **Synchronous Validation:** The controller performs an availability check against the DB before attempting an insertion.
- **Conditional Flow:** The process diverges based on whether the course code is new or exists; existing codes trigger a "Duplicate code" error prompt.
- **Confirmation loop:** Successful insertions return the Primary Key (`course_id`) to the controller to confirm persistent storage.

12 Class Diagram Responsibilities

- **CourseCatalogController:** Orchestrates `addCourseToCatalog()`, `updateCreditValue()`, and `toggleActiveStatus()`.
- **Course:** Maintains the state of individual curriculum items and links to departments.
- **Department:** Validates the origin of the course offerings.

Academic Section & Admin Management System
Department Management Service

Service Documentation — System Design

1 Service Overview

The Department Management Service is a core administrative module responsible for defining the institutional hierarchy of the university. It allows administrators to configure academic departments, assign unique institutional codes (e.g., CSE, ME, EE), and manage school affiliations. This service serves as the foundational data provider for the Faculty and Student portals, ensuring that all academic records are correctly categorized under the proper administrative umbrella.

2 Core Responsibilities

- **Organizational Configuration:** Create, update, and manage the lifecycle of institutional departments.
- **Identity Mapping:** Assign and enforce unique department codes to prevent administrative duplication.
- **Contact Management:** Maintain official institutional contact information (emails/phone) for each department.
- **School Affiliation:** Link departments to their respective higher-level Schools or Faculties.
- **Availability Control:** Toggle the active status of departments to control visibility in registration and grading modules.

3 Architecture & Components

3.1 Internal Modules

- **DeptController:** Handles RESTful requests for department configuration and state changes.
- **DeptService:** Orchestrates business logic, including validation of unique department codes.
- **AdminValidator:** Interacts with the Auth Service to ensure only authorized Administrators can modify data.
- **DeptRepository:** Interfaces with the MySQL database to persist department metadata and school links.
- **AuditLogger:** Records changes to institutional structure for administrative tracking.

4 Department Lifecycle

State	Trigger	Description
Draft	Initial Entry	Department details entered but not yet live for student use.
Active	Admin Activation	Department is live; students can be enrolled in associated programs.
Deactivated	Manual Toggle	Department remains in DB but is hidden from selection menus.
Archived	Historical Flag	Department is no longer operational; kept for legacy record access.

5 API Endpoints

Method	Endpoint	Description
GET	/api/admin/departments	Fetch a list of all institutional departments.
POST	/api/admin/departments	Create a new department with code and school info.
PUT	/api/admin/departments/{id}	Update department contact info or name.
PATCH	/api/admin/departments/{id}/status	Toggle the active_flag for a specific department.

6 Data Model

6.1 Department Database Entry (MySQL)

Field	Type	Description
department_id	INT (PK)	Unique auto-incrementing identifier.
department_code	VARCHAR(20)	Short code (e.g., 'CSE', 'HSS').
department_name	VARCHAR(120)	Full official name of the department.
school_name	VARCHAR(120)	Higher-level school (e.g., 'Engineering').
contact_email	VARCHAR(100)	Official departmental inquiry email.
active_flag	BOOLEAN	Current status (Active/Inactive).

7 Security Considerations

- **RBAC Enforcement:** Only users with the 'Admin' role are permitted to execute POST/PUT/PATCH requests.
- **Data Integrity:** Department codes are unique and indexed to prevent relational errors in course management.
- **State Locking:** Departments with active student enrollments or courses cannot be deleted from the system.

8 Inter-Service Integration

1. **Auth Service:** Verifies the administrator's JWT before allowing structural changes.
2. **Course Service:** Queries this service to fetch department lists for course categorization.

3. **Student Service:** Links student profiles to the department via the ‘department$_id$‘$foreignkey$.

3. **Faculty Service:** Assigns faculty members to their respective departments for workload tracking.

9 Error Handling

Error Code	HTTP	Description
DUPLICATE_DEPT_CODE	409	The department code already exists in the system.
DEPT_NOT_FOUND	404	Requested department ID does not exist.
INVALID_SCHOOL	400	The assigned School name does not match institutional standards.
FORBIDDEN_ACTION	403	Non-admin user attempted to modify institutional config.

10 Configuration Parameters

Parameter	Default	Description
MAX_DEPT_CODE_LEN	10	Character limit for department short codes.
ALLOW_DEACTIVATE_ACTIVE	false	Boolean to prevent deactivation of depts with active students.
DEFAULT_SCHOOL	General	Fallback school name for unassigned departments.

11 Sequence Diagram Observations

- **Configuration Flow:** The Admin submits a form to the DeptController. The Service layer checks for code uniqueness before committing to the Repository.
- **Status Update Flow:** When a department is deactivated, the service triggers a cascading check to identify any active dependencies in the Student/Course modules.

12 Class Diagram Responsibilities

- **Admin (Actor):** Authorized entity capable of triggering configuration methods.
- **Department (Entity):** Core data object containing the institutional metadata.
- **DeptRepository:** Manages persistence logic and relational integrity between depts and programs.
- **AuditLogger:** Provides a non-repudiation layer for institutional structural changes.

Academic Section & Admin Management System
Course Feedback Evaluation Service

Service Documentation — System Design

1 Service Overview

The Course Feedback Evaluation Service is responsible for collecting, anonymizing, and aggregating student reviews regarding course content and faculty teaching performance. It serves as a quality assurance tool for the institute, providing Students with a platform to submit qualitative and quantitative ratings. The system automatically processes these entries to generate course-wise performance reports which are then securely shared with the Head of Department (HOD) for academic review and faculty appraisal.

2 Core Responsibilities

- **Feedback Collection:** Provide secure forms for students to rate courses and faculty on various parameters (e.g., clarity, materials, punctuality).
- **Identity Anonymization:** Strip student identifiers from feedback entries before they are viewed by faculty or HODs to ensure unbiased reporting.
- **Course-Faculty Mapping:** Dynamically link feedback to the specific faculty member teaching a course during a given semester.
- **Aggregation & Analytics:** Calculate average ratings and compile sentiment-based summaries from qualitative comments.
- **HOD Reporting:** Facilitate the automated sharing of final evaluation reports with the respective HODs via the Admin Portal.

3 Architecture & Components

3.1 Internal Modules

- **FeedbackController:** Manages the submission of feedback and requests for report generation.
- **EvaluationService:** The core logic handler that calculates metrics and identifies faculty-course pairings.
- **Anonymizer:** A security module that decouples the 'student$_id$'$fromthefeedbackcontentforprivacy.$
- **ReportGenerator:** Formats aggregated data into structured Course-wise Review documents.

- **FeedbackRepository:** Handles database persistence for raw feedback and generated summaries.
- **NotificationEngine:** Alerts HODs when a new evaluation report is ready for review.

4 Evaluation Lifecycle

State	Trigger	Description
Open	Window Start	Students can submit feedback for their enrolled courses.
Submitted	Form Submission	Feedback is recorded and anonymized in the database.
Processing	Window End	System aggregates ratings and compiles qualitative reviews.
Shared	Admin Action	Finalized reports are pushed to the HOD's dashboard.
Archived	Semester End	Reports are stored for historical faculty performance tracking.

5 API Endpoints

Method	Endpoint	Description
POST	/api/feedback/submit	Submit course-wise faculty evaluation (Student only).
GET	/api/feedback/course/{id}	Retrieve aggregated feedback for a specific course.
GET	/api/feedback/reports	Fetch all pending evaluation reports (HOD/Admin only).
POST	/api/feedback/share-hod	Distribute reports to respective Department Heads.

6 Data Model

6.1 Feedback Database Entry (MySQL)

Field	Type	Description
feedback_id	UUID	Unique identifier for the feedback entry.
course_id	INT (FK)	Reference to the course being evaluated.
faculty_id	INT (FK)	Reference to the faculty member.
rating	INT	Quantitative score (e.g., 1 to 5).
comments	TEXT	Qualitative review/comments from the student.
is_anonymous	BOOLEAN	Flag ensuring identity protection.

7 Security Considerations

- **Anonymity Enforcement:** Student IDs are never stored in the same table as qualitative comments to prevent identification.
- **RBAC:** Faculty can see their own aggregated ratings but cannot see raw comments or student details; HODs see all department data.
- **Integrity:** Only students enrolled in a specific 'course$_id$'$(viatheEnrollmenttable)arepermittedtosubmitfeed$

8 Inter-Service Integration

1. **Course Service:** Provides course codes and faculty assignments for the current semester.
2. **Student Service:** Validates enrollment status before accepting feedback submissions.
3. **Auth Service:** Verifies the HOD role before granting access to performance reports.
4. **Admin Service:** Manages the department-level hierarchy to ensure reports reach the correct HOD.

9 Error Handling

Error Code	HTTP	Description
FEEDBACK_ALREADY_SUBMITTED	409	Student has already provided feedback for this course.
NOT_ENROLLED	403	Student is not registered for the requested course.
INSUFFICIENT_DATA	422	Minimum number of responses not met for report generatio
UNAUTHORIZED_REPORT_ACCESS	401	Faculty attempting to access HOD-level reports.

10 Configuration Parameters

Parameter	Default	Description
MIN_RESPONSES_REQUIRED	5	Minimum students needed before a report is visible.
FEEDBACK_WINDOW_DAYS	14	Duration the feedback portal remains open.
MAX_COMMENT_LENGTH	1000	Character limit for qualitative student comments.

11 Sequence Diagram Observations

- **Feedback Flow:** The Student submits data to the FeedbackController. The Anonymizer strips the ID before the EvaluationService saves the record to prevent tracking.
- **HOD Sharing Flow:** Upon admin trigger, the ReportGenerator compiles course-wise data. The NotificationEngine then sends a secure link to the HOD's dashboard.

12 Class Diagram Responsibilities

- **Student (Actor):** Submits evaluations based on their enrolled course list.
- **HOD (Actor):** Authorized role that reviews and evaluates faculty performance reports.
- **Feedback (Entity):** Stores ratings and comments mapped to Faculty and Course IDs.
- **ReportGenerator:** Aggregates individual entries into a formatted departmental summary.

Academic Section & Admin Management System
Leave Application Service

Service Documentation — System Design

1 Service Overview

The Leave Application Service facilitates the digital submission and tracking of station leave requests for residents. It replaces the traditional manual paperwork system by providing students with a centralized portal to input leave details, including duration, purpose, and destination tracking. The service acts as the primary link between the Student Portal and the Hostel Administration, ensuring that all leave records are verified, time-stamped, and routed to the appropriate Warden for approval.

2 Core Responsibilities

- **Request Submission:** Provide a standardized form for students to input leave metadata (dates, purpose, and destination).
- **Destination Tracking:** Record the precise 'Leave$_{A}ddress$'$wherethestudentwillbelocatedduringtheleaveper$
- **Duration Validation:** Automatically calculate 'Total$_{D}ays$'$andenforceinstitutionallimitsonleavelength.$
- **Status Synchronization:** Maintain real-time status updates (Pending, Approved, Rejected) between the student and admin portals.
- **Verification Routing:** Notify the assigned Warden and Mess Manager once a request is successfully validated.

3 Architecture & Components

3.1 Internal Modules

- **LeaveController:** Handles incoming HTTP requests for leave creation, cancellation, and status lookups.
- **LeaveService:** The core orchestrator that applies business rules, such as checking for overlapping leave dates.
- **WardenNotifier:** Dispatches real-time alerts to the specific Warden assigned to the student's hostel.

- **LeaveRepository:** Manages all database CRUD operations for the ‘Form$_Leave$‘$table$.
- **DocumentService:** (Optional) Handles the generation of digital leave slips for security checkpoints.

4 Leave Application Lifecycle

State	Trigger	Description
Pending	Submission	Request is recorded; awaiting Warden/Admin review.
Approved	Warden Action	Warden grants permission; status updated for Student view.
Rejected	Warden Action	Request denied; student is notified of the reason.
Completed	Return Date	The student has returned to campus; record is finalized.
Cancelled	Student Action	Student withdraws request before administrative review.

5 API Endpoints

Method	Endpoint	Description
POST	/api/leave/apply	Submit a new leave request with all required details.
GET	/api/leave/status/{id}	Fetch the current approval status of a specific request.
GET	/api/leave/history	Retrieve all past leave records for the authenticated student.
POST	/api/leave/cancel/{id}	Withdraw a pending leave request.

6 Data Model

6.1 Leave Database Entry (MySQL)

Field	Type	Description
leave_id	INT (PK)	Unique identifier for the leave application.
roll_no	VARCHAR (FK)	Student identifier linked to the Student table.
from_datetime	DATETIME	Scheduled departure timestamp.
to_datetime	DATETIME	Scheduled return timestamp.
leave_address	TEXT	Destination address and contact during leave.
status	ENUM	Current state (Pending, Approved, Rejected, Completed).

7 Security Considerations

- **Ownership Validation:** Users can only view or cancel leave requests associated with their own ‘roll$_no$‘.
- **Input Sanitization:** Strict validation on the ‘leave$_address$‘$fieldtopreventscriptinjection$.
- **Temporal Integrity:** System prevents the submission of leave requests with backdated departure times.
- **Concurrency:** Prevents multiple active leave requests for the same date range per student.

8 Inter-Service Integration

1. **Student Service:** Fetches the student's current hostel and room assignment for the Warden.
2. **Auth Service:** Validates the student's session and provides the ‘roll$_n o$‘$context$.
2. **Hostel Admin Service:** Routes the application to the specific Warden's dashboard.
3. **Notification Service:** Sends push/email notifications for approval or rejection alerts.

9 Error Handling

Error Code	HTTP	Description
OVERLAPPING_LEAVE	409	Student already has a request for this date range.
INVALID_DURATION	400	The 'To Date' is earlier than the 'From Date'.
MISSING_DESTINATION	422	Destination address field was left empty.
UNAUTHORIZED_ACCESS	403	Student attempting to view another student's request.

10 Configuration Parameters

Parameter	Default	Description
MAX_LEAVE_DAYS	15	Maximum consecutive days allowed for a single leave.
ADVANCE_NOTICE_HRS	24	Minimum hours before departure a request must be filed.
AUTO_ARCHIVE_DAYS	180	Days after which completed leaves are moved to history.

11 Sequence Diagram Observations

- **Submission Flow:** The student submits credentials and data to the LeaveController. The LeaveService validates the date logic and commits the record with a ‘Pending‘ status.
- **Warden Interaction:** Once committed, the WardenNotifier immediately updates the Warden's administrative dashboard, moving the request into the review queue.

12 Class Diagram Responsibilities

- **LeaveRequest (Entity):** Represents the core data structure including ‘roll$_n o$‘, $dates, and$‘$leave_a ddress$‘.
- **Warden (Actor):** The administrative role responsible for invoking the ‘approve()‘ or ‘reject()‘ methods.
- **LeaveService:** Handles complex logic like duration calculations and constraint checking.
- **LeaveRepository:** Provides a clean interface for data persistence and multi-attribute queries.

Academic Section & Admin Management System
No Dues Clearance Service

Service Documentation — System Design

1 Service Overview

The No Dues Clearance Service is a specialized administrative module designed to handle the graduation and exit workflows for students leaving the institute or hostel. It digitalizes the multi-step "No Dues" process, which traditionally involves physical signatures from various departments. This service allows students to initiate a clearance request, tracks the mandatory inspections by the Caretaker, and facilitates the final administrative sign-off by the Warden to ensure all institutional assets are returned and financial liabilities are settled.

2 Core Responsibilities

- **Clearance Initiation:** Provide students with a portal to formally request "No Dues" for graduation or permanent exit.
- **Asset Verification:** Coordinate with the Caretaker role to confirm that room furniture, electrical appliances, and keys have been returned.
- **Multi-Stage Sign-off:** Enforce a sequential approval logic where the Warden's final signature is only permitted after the Caretaker's clearance.
- **Digital Certification:** Generate a verifiable digital "No Dues" status that can be consumed by the Academic Section for issuing transcripts.
- **Dues Tracking:** (Optional) Interface with the Mess and Library modules to check for outstanding fines or subscriptions.

3 Architecture & Components

3.1 Internal Modules

- **ClearanceController:** Manages incoming requests for clearance and provides status updates to the Student.
- **SignatureProcessor:** Orchestrates the sequential signing logic between Caretakers and Wardens.
- **InspectionService:** Provides the Caretaker with a checklist of assets linked to the student's specific room ID.

- **NoDuesRepository:** Interfaces with the ‘No$_{D}ues_{F}orm$‘$tabletopersistsignaturetimestampsandstatus$.

- **ExitNotifier:** Alerts the Academic Section once the final Warden signature is recorded.

4 Clearance Lifecycle

State	Trigger	Description
Requested	Student Application	Initial state; record created in the system.
Caretaker-Signed	Asset Inspection	Staff confirms physical assets are in order.
Warden-Signed	Final Review	Warden provides final administrative clearance.
Cleared	Dual Signatures	Process complete; student is eligible for transcript release.
Rejected	Discrepancy Found	Clearance halted due to missing assets or unpaid dues.

5 API Endpoints

Method	Endpoint	Description
POST	/api/nodues/apply	Student initiates the exit clearance process.
GET	/api/nodues/status	Fetch the current progress (signatures pending).
PATCH	/api/nodues/caretaker-sign	Caretaker records room inspection results.
PATCH	/api/nodues/warden-sign	Warden provides the final graduation sign-off.

6 Data Model

6.1 No Dues Database Entry (MySQL)

Field	Type	Description
id	INT (PK)	Unique identifier for the clearance form.
roll_no	VARCHAR (FK)	Student identifier linked to the Student profile.
caretaker_sign	VARCHAR	Digital signature/ID of the inspecting Caretaker.
warden_sign	VARCHAR	Digital signature/ID of the approving Warden.
status	ENUM	Current state (Requested, Signed, Cleared, Rejected).

7 Security Considerations

- **Non-Repudiation:** Signatures are tied to the staff’s unique Institute ID and are timestamped to prevent post-dated approvals.

- **Strict Sequence:** The API prevents ‘warden$_{s}ign$‘$frombeingpopulatedif$‘$caretaker_{s}ign$‘$isstillnull$.

- **Audit Logging:** Every modification to the clearance status is logged for administrative oversight.

- **Account Status:** System checks if the ‘Account$_{S}tatus$‘$is'Active'beforeallowingaclearancerequest$.

8 Inter-Service Integration

1. **Student Portal:** Provides the interface for application and status tracking.
2. **Admin Portal:** Hosts the dashboard for Caretakers and Wardens to review and sign requests.
3. **Hostel Service:** Queries room and asset data to generate the Caretaker's inspection checklist.
4. **Academic Service:** Automatically receives a notification once a student is "Cleared" to proceed with degree issuance.

9 Error Handling

Error Code	HTTP	Description
UNFINISHED_INSPECTION	422	Warden attempted to sign before the Caretaker.
PENDING_DUES	403	Clearance cannot proceed until mess/library fines are paid.
ALREADY_CLEARED	409	Student has already completed the No Dues process.
INVALID_STAFF_ROLE	401	Non-hostel staff attempted to sign the clearance form.

10 Configuration Parameters

Parameter	Default	Description
CLEARANCE_WINDOW_DAYS	30	Timeframe before graduation when portal opens.
REQUIRE_MESS_CLEARANCE	true	Toggle for mandatory mess bill verification.
NOTIFY_ACADEMIC_SECTION	true	Automated alert to registrar upon final sign-off.

11 Sequence Diagram Observations

- **Verification Flow:** The Student submits the application. The system then routes the task sequentially. The Caretaker must physically inspect before the digital "Sign" button is enabled in their portal.
- **Finalization Flow:** Upon the Warden's signature, the record status moves to 'Cleared', triggering an asynchronous update to the Academic Section's database.

12 Class Diagram Responsibilities

- **NoDuesForm (Entity):** Core data object tracking the 'roll$_n o$'$andthedual-signaturestate$.
- **Caretaker (Actor):** Performs the primary physical inspection and initial sign-off.
- **Warden (Actor):** Acts as the final authority to grant institutional clearance.
- **SignatureProcessor:** Implements the business rules governing the order of approvals.

Academic Section & Admin Management System
Hostel Transfer Request Service

Service Documentation — System Design

1 Service Overview

The Hostel Transfer Request Service manages the formal process for students seeking to change their residential building or room. This service digitalizes a complex workflow that requires coordination between the student, the Outgoing Warden (current hostel), and the Incoming Warden (target hostel). It allows students to submit ranked preferences for new hostels along with a formal justification, while providing administrators with a platform to review eligibility, verify room availability, and execute the physical transfer in the system.

2 Core Responsibilities

- **Transfer Application:** Provide a structured form for students to submit a 'Reason' for transfer and select up to three 'Target$_{Preferences}$'.

- **Inter-Hostel Coordination:** Facilitate a dual-approval workflow requiring a "Release" from the current hostel and an "Acceptance" from the target hostel.
- **Capacity Validation:** Interface with the Hostel Infrastructure module to ensure target rooms have available slots before final approval.
- **Eligibility Checking:** Verify that the student has no pending disciplinary actions or outstanding mess dues before initiating a move.
- **Inventory Handover:** Trigger notifications for room asset inspections as part of the move-out/move-in process.

3 Architecture & Components

3.1 Internal Modules

- **TransferController:** Manages the lifecycle of the transfer request from submission to final execution.
- **RoutingService:** Identifies and routes approval tasks to the correct Wardens based on the student's current and preferred hostels.
- **AvailabilityChecker:** Queries the Room Table to provide real-time occupancy data for the Incoming Warden's review.

- **TransferRepository:** Handles data persistence for the ‘Form$_{H}ostel_{C}hange$‘$table$.
- **NotificationEngine:** Dispatches push alerts to students and email summaries to the respective administrative offices.

4 Transfer Lifecycle

State	Trigger	Description
Pending	Student Submission	Request is created; awaiting Outgoing Warden review.
Released	Outgoing Approval	Current Warden approves the exit; routed to Incoming Warden.
Accepted	Incoming Approval	Target Warden approves the entry; room is allocated.
Completed	Physical Move	Student checked into new room; records updated.
Rejected	Warden Denial	Transfer denied at either stage; justification sent to student.

5 API Endpoints

Method	Endpoint	Description
POST	/api/hostel/transfer/apply	Student submits a request with preferences and reason.
GET	/api/hostel/transfer/status	Retrieve the current stage of the transfer request.
PATCH	/api/admin/transfer/release	Outgoing Warden approves the release of the student.
PATCH	/api/admin/transfer/accept	Incoming Warden approves the entry and assigns a room.

6 Data Model

6.1 Hostel Transfer Entry (MySQL)

Field	Type	Description
request_id	INT (PK)	Unique identifier for the transfer request.
roll_no	VARCHAR (FK)	Student identifier linked to the central student profile.
current_hostel	VARCHAR	ID of the hostel the student is currently assigned to.
target_pref_1	VARCHAR	First preference for the new hostel assignment.
reason	TEXT	Formal justification for the requested transfer.
status	ENUM	Current state (Pending, Released, Accepted, Rejected).

7 Security Considerations

- **Role-Based Routing:** The API ensures that only the Warden assigned to the ‘current$_{h}ostel$‘$cantriggerthe$‘r
- **Integrity Enforcement:** System prevents a student from having multiple active transfer requests simultaneously.
- **Audit Trails:** All changes to the transfer status are logged with the ID of the approving staff for accountability.
- **Data Privacy:** Justifications are visible only to the involved Wardens and the student.

8 Inter-Service Integration

1. **Hostel Service:** Fetches real-time room occupancy and gender-type restrictions for target hostels.
2. **Admin Portal:** Provides the interface for Wardens to review and decide on applications.
3. **Student Portal:** Allows students to apply and monitor their request's progress.
4. **Mess Service:** Checks for outstanding monthly mess bills before allowing the transfer to proceed.

9 Error Handling

Error Code	HTTP	Description
NO_VACANCY	422	The requested target hostel has no available rooms.
PENDING_DUES	403	Transfer blocked due to unpaid mess or hostel fees.
INVALID_WARDEN	401	Staff attempting to approve a request for a different hostel.
TRANSFER_WINDOW_CLOSED	400	Transfer requests are only allowed during specific semester dates.

10 Configuration Parameters

Parameter	Default	Description
MAX_PREFERENCES	3	Maximum number of target hostels a student can select.
TRANSFER_EXPIRY_DAYS	7	Days a request remains valid before automatic cancellation.
REQUIRE_MESS_CLEARANCE	true	Enforces zero-dues check before outgoing approval.

11 Sequence Diagram Observations

- **Submission Flow:** The Student submits the application. The system validates the 'roll$_n o$'$andcurrentassign$

- **Approval Coordination:** The process is sequential. The Incoming Warden cannot "Accept" a student until the Outgoing Warden has formally "Released" them in the system.

12 Class Diagram Responsibilities

- **HostelTransferRequest (Entity):** Stores the core metadata, preferences, and the state of the transfer.
- **Warden (Actor):** Specialized admin role that invokes release/acceptance methods based on their 'assigned$_h ostel_i d$'.
- **RoutingService:** Crucial for determining which administrative portals should receive the pending task.

- **AvailabilityChecker:** Logic layer that prevents over-allocation of rooms during the transfer process.

Academic Section & Admin Management System
Complaint Filing & Tracking Service

Service Documentation System Design

1 Service Overview

The Complaint Filing & Tracking Service is responsible for capturing, processing, and managing facility-related grievances raised by students. It serves as the digital interface between the student body and the maintenance department, ensuring that issues like faulty appliances or carpentry needs are documented and routed to the appropriate Warden or Administrator for resolution.

2 Core Responsibilities

- **Issue Intake:** Facilitate the selection of categories (Electrical, Carpentry) and collection of location details.
- **Evidence Management:** Support the attachment of multiple images (`image_1`, `image_2`, `image_3`) to provide visual context for the fault.
- **Automated Alerting:** Trigger real-time notifications to the Warden/Admin Portal upon successful database insertion.
- **Status Lifecycle:** Manage the transition of complaints from 'Pending' to 'Resolved' through administrative action.
- **Student Tracking:** Generate unique Complaint IDs to allow students to monitor progress via the Communication System.

3 Architecture & Components

3.1 Internal Modules

- **Student Portal (Frontend):** Handles user input, category selection, and image uploads.
- **Backend Server:** Validates student status and coordinates data mapping to maintenance tables.
- **Complaint Entity:** The primary data model containing details like `avail_slot`, `room_id`, and `date_origin`.
- **CommunicationsSystem (Module 6):** Handles the dispatch of alerts and notification alerts to stakeholders.

4 Authentication & Lifecycle

State	Trigger	Description
Requested	"Raise Complaint"	Student initiates the filing process on the portal.
Validated	Status Check	Backend verifies the student's "Active" status.
Persisted	INSERT INTO DB	Complaint is saved with 'Pending' status.
Notified	Trigger Alert	Warden receives a notification for [Room_ID].
Active	Success Return	Student tracks the issue using the generated ID.

5 API Endpoints

Method	Endpoint	Description
POST	/api/complaints/raise	Submit details (Roll_No, Category, Slot, Images).
GET	/api/complaints/view	Retrieve history via Student/Warden portal.
PATCH	/api/complaints/resolve	Update status to 'Resolved' by Admin/Warden.

6 Data Model

6.1 Complaint Table (MySQL)

Field	Type	Description
complaint_id	INT	Primary unique identifier for the issue.
category	ENUM	Issue type (e.g., Electrical, Carpentry).
avail_slot	ENUM	Morning or Evening availability of the resident.
status	ENUM	Current state (Pending, In-Progress, Resolved).
image_1	STRING	Path or link to the first evidence photo.

7 Security Considerations

- **Identity Verification:** The backend validates "active student status" before allowing a POST request.
- **Data Mapping:** Inputs are strictly mapped to Table 8 (Maintenance) to ensure data integrity.
- **RBAC:** Only Wardens have the authority to review pending complaints and assign caretakers.

8 Inter-Service Integration

1. Student submits the form via the Student Portal Frontend.
2. Backend Server validates the request and maps data to the database.
3. Database returns a unique `Complaint_ID` upon successful insertion.
4. Communication System triggers an alert to the Warden/Admin Portal.
5. Warden receives an alert specific to the student's [Room_ID].

9 Error Handling

Error Code	HTTP	Description
INACTIVE_STUDENT	403	Student is not currently active in the system.
UPLOAD_FAILED	400	Images could not be processed or are missing.
DB_WRITE_ERROR	500	Failure to insert complaint into Complaint_Table.

10 Configuration Parameters

Parameter	Default	Description
MAX_IMAGES	3	Maximum number of photos allowed per complaint.
SLOT_TYPES	M, E	Available time slots (Morning, Evening).
ALERT_TARGET	Warden	Default role receiving new complaint notifications.

11 Sequence Diagram Observations

- **Input Flow:** The student provides Room_ID and Location_Detail (e.g., 'Tube light in Room 101').
- **Status Logic:** New complaints are automatically initialized with `Status='Pending'`.
- **Integration:** The process explicitly links to "Module 6" for tracking status after the initial filing.

12 Class Diagram Responsibilities

- **Warden:** Responsible for `reviewPendingComplaints()` and `assignCaretaker()`.
- **Complaint:** Encapsulates `validateDetails()` and `updateStatus()`.
- **Room:** Provides `getResidentDetails()` to link the issue back to a specific occupant.

Academic Section & Admin Management System
Complaint Status Tracking Service

Service Documentation System Design

1 Service Overview

The Complaint Status Tracking Service provides real-time visibility into the resolution lifecycle of facility issues. It allows students to monitor the progress of their filed complaints through a dedicated dashboard and enables Wardens to update the status of these issues as they move from initial filing to final resolution.

2 Core Responsibilities

- **Status Monitoring:** Allow students to fetch and view a list of active and resolved complaints associated with their Roll Number.
- **Lifecycle Management:** Support the transition of complaints through defined states: PENDING, IN_PROGRESS, RESOLVED, and REJECTED.
- **Administrative Updates:** Provide Wardens with the ability to update status and add resolution notes to specific complaints.
- **Automated Notifications:** Trigger push notifications and email alerts to students whenever a status change occurs.
- **Audit Trail:** Maintain a timeline of updates for each complaint to ensure transparency in the maintenance process.

3 Architecture & Components

3.1 Internal Modules

- **ComplaintDashboard:** Acts as the primary interface for students to filter and refresh the status of their complaints.
- **NotificationSystem (Module 6):** Handles the delivery of status updates via `pushStatusUpdate()` and `sendEmailAlert()`.
- **Backend Server:** Coordinates requests between the frontend portals and the database, ensuring valid state transitions.
- **Complaint Entity:** Stores the core state including `status`, `location_detail`, and `date_origin`.

4 Tracking Lifecycle

State	Trigger	Description
PENDING	Initial Filing	Complaint is recorded and awaiting review.
IN_PROGRESS	Warden Action	Maintenance has been assigned or started.
RESOLVED	Work Completed	Issue is fixed and marked as resolved by Warden.
REJECTED	Review Decision	Complaint is deemed invalid or duplicate.

5 API Endpoints

Method	Endpoint	Description
GET	/api/complaints/student/{Roll_No}	Fetch all complaints for a specific student.
PATCH	/api/admin/complaints/{ID}	Update status (e.g., to 'In-Progress' or 'Resolved').
GET	/api/complaints/{ID}/details	Retrieve full timeline and resolution notes.

6 Data Model

6.1 Complaint Entry (MySQL)

Field	Type	Description
complaint_id	INT	Primary unique identifier.
roll_no	VARCHAR	Student ID associated with the complaint.
status	ENUM	Current lifecycle stage (PENDING, etc.).
images[3]	STRING ARRAY	Collection of evidence photos.
date_origin	TIMESTAMP	Creation timestamp.

7 Security Considerations

- **Authorized Access:** Students can only view complaints associated with their own Roll Number.
- **Role-Based Updates:** Only users with the 'Warden' role can trigger PATCH requests to change complaint status.
- **Data Consistency:** Database transactions ensure that `Status Updated` alerts are only sent after successful SQL commits.

8 Inter-Service Integration

1. Student Portal requests complaint list from the Backend Server.
2. Backend queries the `Complaint_Table` using the Student's unique ID.
3. Warden Portal sends status updates (PATCH) to the Backend.
4. Backend triggers the Notification System (Module 6) upon successful update.
5. Student receives a real-time push notification regarding the change.

9 Error Handling

Error Code	HTTP	Description
COMPLAINT_NOT_FOUND	404	The requested Complaint ID does not exist.
UNAUTHORIZED_UPDATE	401	Non-admin user attempting to change status.
INVALID_STATE_TRANSITION	400	Attempting to move from 'Resolved' back to 'Pending'.

10 Configuration Parameters

Parameter	Default	Description
NOTIF_EMAIL_ENABLED	TRUE	Flag to enable/disable automated email alerts.
DASHBOARD_PAGE_SIZE	20	Number of complaints displayed per page.
REJECT_REASON_REQ	TRUE	Requirement for notes when status is 'REJECTED'.

11 Sequence Diagram Observations

- **Dual Update Flow:** The diagram illustrates a two-stage update process: moving to 'In-Progress' then to 'Resolved'.
- **Real-time Feedback:** The "System sends Push Notification" step occurs immediately after the database record is updated to 'In-Progress'.
- **Final Review:** The student performs a "Refresh Dashboard" to view the final "Resolved - Maintenance Completed" status.

12 Class Diagram Responsibilities

- **Student:** Monitors the lifecycle through `viewComplaintDashboard()`.
- **Warden:** Authorized to `updateStatus()` and `addResolutionNotes()`.
- **Complaint Dashboard:** Responsible for `filterByStatus()` and `refreshStatus()`.
- **NotificationSystem:** Decouples communication logic via `pushStatusUpdate()`.

Academic Section & Admin Management System
Leave Approval & Management Service

Service Documentation System Design

1 Service Overview

The Leave Approval & Management Service is a critical administrative module designed to handle student leave requests within the hostel ecosystem. It provides a structured workflow for Wardens to review, approve, or reject pending leave applications, ensuring that student movement is documented and authorized by the appropriate hostel authorities.

2 Core Responsibilities

- **Request Retrieval:** Fetch and display all leave applications currently in a 'Pending' state for administrative review.
- **Decision Processing:** Enable Wardens to execute approval or rejection logic, including mandatory signature data for approvals.
- **Status Orchestration:** Update the lifecycle of a request through states: PENDING, APPROVED, REJECTED, and COMPLETED.
- **Automated Alerting:** Notify students of status changes and alert security personnel upon leave approval.
- **Validation:** Calculate leave duration and verify student identification (Roll No) during the decision process.

3 Architecture & Components

3.1 Internal Modules

- **Admin Portal (Frontend):** Interface used by Wardens to log in and select "Pending Leave Requests".
- **Backend Server:** Orchestrates the API logic between the portal and the `Form_Leave` database table.
- **LeaveRequest (Entity):** Core data model containing leave address, purpose, and total days.
- **NotificationSystem (Module 6):** Handles external communications, including student notifications and security alerts.

4 Leave Approval Lifecycle

State	Trigger	Description
PENDING	Application Submission	Request is saved and awaits Warden review.
APPROVED	Warden Signature	Request is authorized; Security is alerted.
REJECTED	Warden Decision	Request is denied with a provided reason.
COMPLETED	Leave Conclusion	The leave period has successfully ended.

5 API Endpoints

Method	Endpoint	Description
GET	/api/admin/leaves/pending	Retrieves all records where Status = 'Pending'.
POST	/api/admin/leaves/approve	Submits approval signature and updates status.
POST	/api/admin/leaves/reject	Submits rejection reason and updates status.

6 Data Model

6.1 Leave Database Entry (MySQL)

Field	Type	Description
leave_id	INT	Primary identifier for the leave request.
roll_no	STRING	Institutional ID of the student applicant.
from_date_time	DATETIME	Scheduled start of the leave.
status	ENUM	Current state (Pending, Approved, etc.).
approvals	STRING	Encrypted or stringified signature data.

7 Security Considerations

- **RBAC Enforcement:** Only users with the 'Warden' role assigned to a specific hostel (`assigned_hostel_id`) can approve leaves.
- **Electronic Signatures:** Approvals require a `signature` parameter to be recorded via `recordApprovals()`.
- **Audit Trail:** Status transitions are logged in the `Form_Leave` table for administrative auditing.

8 Inter-Service Integration

1. Warden authenticates and requests pending leaves via the Admin Portal.
2. Backend Server executes a SQL SELECT query against the `Form_Leave` Table.
3. Upon decision, the Warden object triggers `updateStatus()` on the LeaveRequest entity.
4. The LeaveRequest entity sends signals to the Notification System (Module 6).
5. Notification System executes `notifyStudent()` and `alertSecurity()` based on the outcome.

9 Error Handling

Error Code	HTTP	Description
NO_PENDING_LEAVES	200 (Empty)	No records found with Status='Pending'.
INVALID_LEAVE_ID	404	The specified leave_id does not exist in the database.
MISSING_SIGNATURE	400	Approval attempted without providing required signature

10 Configuration Parameters

Parameter	Default	Description
MAX_LEAVE_DAYS	15	Upper limit for a single leave duration.
NOTIFY_SECURITY	TRUE	Toggle for sending alerts to the security module.
AUTO_REJECT_DAYS	7	Days after which a pending request is auto-rejected.

11 Sequence Diagram Observations

- **Query Logic:** The system uses a filtered query (`WHERE Status = 'Pending'`) to minimize data transfer to the Admin Portal.
- **Synchronous Fetch:** The "Return list of requests" is a synchronous response from the database to the backend server.
- **Warden Log-in:** The workflow assumes the Warden is already logged in to review hostel-specific requests.

12 Class Diagram Responsibilities

- **Warden:** Core actor responsible for `approveLeave()` and `signLeaveForm()`.
- **LeaveRequest:** Responsible for self-encapsulation tasks like `getDuration()`.
- **NotificationSystem:** Decouples the messaging logic from the business approval logic.
- **LeaveStatus:** An enumeration ensuring strict type safety for request states.

Academic Section & Admin Management System
HMC Member & Minutes Management Service

Service Documentation System Design

1 Service Overview

The HMC Member & Minutes Management Service facilitates the documentation and dissemination of hostel-specific administrative decisions. It allows Hostel Management Committee (HMC) members, such as Student Secretaries, to record Minutes of Meetings (MoM) and submit them for administrative review. The service acts as a formal channel for transparency, ensuring that meeting outcomes are reviewed by the Warden before being published to the general student body.

2 Core Responsibilities

- **Role Verification:** Validate the specific HMC position (e.g., Mess Secy, Sports Secy) of a user before granting management access.
- **Minutes Recording:** Capture meeting data including date, agenda, and detailed discussion text.
- **Review Workflow:** Facilitate a "Pending Review" state where minutes are alerted to the Warden for approval.
- **Official Publication:** Convert approved minutes into global hostel announcements accessible to all students.
- **Tenure Management:** Track HMC member details including tenure years and assigned hostel IDs.

3 Architecture & Components

3.1 Internal Modules

- **HMCMember (Entity):** Represents the specialized student role authorized to conduct meetings and record minutes.
- **MeetingMinutes (Core Entity):** Manages the data lifecycle of a meeting record, including its content and publication status.
- **Backend Server:** Orchestrates role verification in Table 7 (HMC_Member) and manages data insertion into the Announcements table.
- **CommunicationsSystem (Module 6):** Dispatches notifications to Wardens for review and global alerts to students upon publication.

4 Minutes Management Lifecycle

State	Trigger	Description
Verified	Role Check	Backend confirms user is an authorized HMC member.
Submitted	POST Request	Minutes are saved as 'Pending' in the Announcements table.
Alerted	Notif. Service	Warden is notified that new minutes require review.
Published	PATCH Request	Warden approves; minutes become visible to all students.

5 API Endpoints

Method	Endpoint	Description
GET	/api/hmc/verify	Check user position and grant management access.
POST	/api/hmc/minutes	Submit new MoM text, date, and agenda.
PATCH	/api/hmc/minutes/publish	Warden-only endpoint to approve and publish minutes.

6 Data Model

6.1 HMC & Minutes Entry (MySQL)

Field	Type	Description
hmc_id	INT	Unique identifier for the HMC member.
position	ENUM	Role (SPORTS_SECY, CULTURAL_SECY, etc.).
meeting_id	INT	Primary key for the specific meeting record.
is_published	BOOLEAN	Flag determining visibility to the student portal.
minutes_text	TEXT	Full content of the meeting discussion.

7 Security Considerations

- **Role-Based Access Control:** Access to the "Update Minutes" portal is strictly gated by a database check of the user's Roll Number in the HMC_Member table.
- **Warden Review Gate:** The system prevents direct publication by students; a PATCH request from the Warden is mandatory to set `is_published` to true.
- **Hostel Isolation:** Announcements are linked to a specific `hostel_id` to ensure notifications reach only the relevant resident group.

8 Inter-Service Integration

1. HMC Member requests access via the Admin/HMC Portal Frontend.
2. Backend Server queries the HMC_Member database to return the user's specific Role/Position.
3. Upon MoM submission, the Backend inserts data into the Announcements table with `Type='HMC_Minutes'`.
4. The Notification Service alerts the Warden via the Communications System.
5. Upon Warden approval, the Communications System triggers a Global Hostel Notification.

9 Error Handling

Error Code	HTTP	Description
UNAUTHORIZED_ROLE	403	User Roll_No not found in HMC_Member table.
MOM_SUBMISSION_FAIL	400	Malformed meeting data or missing mandatory fields.
WARDEN_REJECTED	401	Warden denied publication; minutes remain unpublished.

10 Configuration Parameters

Parameter	Default	Description
ANNOUNCEMENT_TYPE	HMC_Minutes	Standard category for HMC-related posts.
MAX_TENURE_YEARS	1	Default duration for an HMC member's role.
AUTO_ARCHIVE_DAYS	365	Period after which old minutes are archived.

11 Sequence Diagram Observations

- **Verification Loop:** The system performs a verified role check against Table 7 before granting the "Minutes Management" view.
- **Warden Publication:** The diagram explicitly shows that the Warden "Approves Minutes for Publication" as a separate step from the student's submission.
- **Notification Recipients:** Two distinct notifications are triggered: one to the Warden for "Review" and one to Hostel Students for "Posted".

12 Class Diagram Responsibilities

- **HMCMember:** Responsible for `conductMeeting()`, `recordMinutes()`, and `requestAnnouncem`
- **MeetingMinutes:** Encapsulates data state and the `markAsPublished()` logic.
- **Warden:** Authorized to `reviewMinutes()` and `approveForPublication()`.
- **CommunicationsSystem:** Manages the announcement list and triggers group notifications.

Academic Section & Admin Management System
Facility & Asset Management Service

Service Documentation System Design

1 Service Overview

The Facility & Asset Management Service is responsible for overseeing the physical infrastructure and equipment within the hostel ecosystem. It manages the lifecycle of hostel facilities (like Canteens and Gyms) and individual assets, ensuring operational readiness through maintenance tracking and tender management.

2 Core Responsibilities

- **Facility Oversight**: Manage operational status and tender details for hostel-provided services (Laundry, Music, etc.).
- **Asset Tracking**: Monitor the operative status (Full, Partial, Not Working) of specific equipment and infrastructure items.
- **Maintenance Logging**: Maintain historical maintenance records and operative logs for all facility assets.
- **Complaint Resolution**: Link reported faulty items from students to specific assets for streamlined administrative processing.
- **Inventory Management**: Update inventory status and facility types dynamically based on hostel needs.

3 Architecture & Components

3.1 Internal Modules

- **Admin (Warden/Staff)**: The primary orchestrator responsible for updating inventory and processing facility complaints.
- **Hostel_Facility**: Represents large-scale services, managing tender holders and contract expiry dates.
- **Facility_Assets**: Represents individual items within a hostel, tracking their category and operative status.
- **Complaint_Table**: Handles the intake of reports regarding faulty items, referencing specific assets for repair.

4 Management Lifecycle

State	Trigger	Description
Operational	System Init	Asset is fully functional and available for use.
Faulty	Complaint Filed	Student reports an issue; linked to specific asset ID.
Under Repair	Admin Action	Status updated to 'Partial' or 'Not_Working' during fix.
Resolved	Admin Update	Operative status restored to 'Full' after maintenance.

5 API Endpoints

Method	Endpoint	Description
GET	/api/facility/dashboard	View all facilities and asset status overview.
PATCH	/api/facility/asset/status	Update asset operative status (Full/Partial/Not_Workin
POST	/api/facility/tender/update	Update contract expiry and tender holder details.
GET	/api/facility/maintenance/logs	Retrieve maintenance history for a specific asset ID.

6 Data Model

6.1 Asset & Facility Tables (MySQL)

Field	Type	Description
facility_id	INT (PK)	Unique identifier for the hostel facility.
asset_id	INT (PK)	Unique identifier for a specific asset item.
operative_status	ENUM	Full, Partial, or Not_Working.
contract_expiry	DATE	Expiration of the current service tender.
is_operational	BOOLEAN	Global flag for facility availability.

7 Security Considerations

- **Access Control**: Only users with 'Admin' or 'Warden/Staff' roles can update inventory or tender details.
- **Audit Logging**: Every status change (`updateOperativeStatus`) is recorded in maintenance logs for accountability.
- **Data Integrity**: Complaint resolution must link to a valid `asset_id` via `linkToAsset()`.

8 Inter-Service Integration

1. Admin requests the Facility Dashboard to view current hostel health.
2. Student reports a fault, creating an entry in the `Complaint_Table`.
3. Admin processes the complaint, linking it to a specific entry in `Facility_Assets`.
4. Admin updates the operative status, which automatically reflects in the Hostel overview.
5. System triggers a status refresh for the `Hostel_Facility` to reflect operational changes.

9 Error Handling

Error Code	HTTP	Description
INVALID_ASSET_ID	404	Referenced asset for complaint link does not exist.
EXPIRED_TENDER	403	Action attempted on a facility with an expired contract.
INVALID_STATUS	400	Attempted to set a status not defined in `OperativeStatus`.

10 Configuration Parameters

Parameter	Default	Description
FACILITY_TYPES	CANTEEN, etc.	Enumerated types of supported hostel facilities.
OPERATIVE_LEVELS	FULL, etc.	Valid states for asset health tracking.
NOTIF_THRESHOLD	PARTIAL	Level at which maintenance alerts are triggered.

11 Sequence Diagram Observations

- **Monitoring Flow**: The Admin monitors status by viewing the dashboard, which aggregates data from both Facilities and Assets.
- **Resolution Loop**: Resolving a report involves `reviewComplaint()` followed by `resolveIssue()` within the table entity.
- **Entity Composition**: The Hostel entity acts as a central hub, containing multiple assets and providing multiple facilities.

12 Class Diagram Responsibilities

- **Admin**: Core actor for dashboard views and processing facility complaints.
- **Facility_Assets**: Responsible for status updates and maintenance log retrieval.
- **Hostel_Facility**: Manages operational toggles and tender-holder details.
- **Complaint_Table**: Manages student reports and links them to physical inventory for repair.